

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет Механико - математический

Кафедра Программирования

Направление подготовки Математика и компьютерные науки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Пьянзин Богдан Олегович

(Фамилия, Имя, Отчество автора)

Тема работы Методы адаптивного планирования запросов в реляционных СУБД

«К защите допущена»

Заведующий кафедрой

к.ф - м.н

Емельянов П.Г./.....

(фамилия , И., О.) / (подпись, МП)

«.....».....20...г.

Научный руководитель

к.ф - м.н

Старший преподаватель кафедры
программирования

Пономарев Д.К./.....

(фамилия , И., О.) / (подпись, МП)

«.....».....20...г.

Дата защиты: «.....».....20...г.

Новосибирск, 20__

Аннотация

Работа посвящена разработке новых эвристических методов планирования запросов в реляционных СУБД. В литературе известно достаточно много алгоритмов планирования, но в промышленных приложениях известны лишь базовые подходы, опирающиеся на динамическое программирование, жадные и генетические алгоритмы и их простейшие эвристические комбинации, позволяющие выбирать тот или иной алгоритм в зависимости от числа таблиц в запросе. Данная работа преследует цель получить глубокое понимание того, как разные алгоритмы возможно комбинировать в ходе планирования запроса, делая переключение между алгоритмами в зависимости от топологии и сложности графа запроса. Ожидаемым результатом работы является решение оптимизационной задачи планирования с помощью новых эвристик, реализация решения в виде модификации планировщика в ядре реляционной СУБД с открытым кодом и экспериментальная оценка на известных промышленных бенчмарках.

Постановка задачи

Дано:

- Реляционная СУБД PostgreSQL вместе с её планировщиком и стоимостной моделью;
- SQL-запросы промышленных бенчмарков;
- Статистики таблиц(число строк, гистограммы значений, количество различных значений, наличие индексов);
- Аппаратные ресурсы исполнения, отражённые в параметрах

стоимостной модели.

- Данные, на которых выполняются запросы. Представлены таблицами.

Требуется получить:

- Новый метод выбора порядка соединений, реализованный в ядре PostgreSQL и интегрированный в конвейер планирования;
- Экспериментальная оценка на промышленных бенчмарках;
- Сравнение нового подхода со стандартным динамическим программированием PostgreSQL, а также с известными альтернативами.

Содержание

1. Введение	6
2. Формальная постановка задачи	13
3. Предшествующие работы	15
3.1. Задача выбора порядка соединений	15
3.2. Принадлежность NP	15
3.3. NP-полнота	16
3.4. Пространство поиска планов	16
3.5. Жадный подход к планированию	16
3.6. Динамическое программирование по размеру подпланов (DPsize)	18
3.7. Динамическое программирование по подмножествам (DPsub)	20
3.8. Динамическое программирование по парам связных подграфов (DPccp)	22
3.9. DPhyp: динамическое программирование на гиперграфах . . .	24
3.9.1. Гиперграф и csg-cmp-пары	25
3.9.2. Идея перечисления	25
3.9.3. DPhyp	26
3.10. Генетический подход	27
3.11. Адаптивная оптимизация больших запросов соединения . . .	28
3.12. Архитектура DP планировщика PostgreSQL	30
3.13. Оценки в модели стоимости	32
4. Ход исследования	35
4.1. Окружение	35

4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения	37
4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии	41
4.4. Подход 3. DPhyr (реализация с открытым исходным кодом) .	44
4.5. Подход 4. Модифицированный DPhyr (DPhyrMod)	47
4.6. Подход 5. Связывание якорных участков	52
5. Заключение	63
5.1. Полученные результаты	63
5.2. Направления дальнейшей работы	65
Приложение А. Результаты при стандартных оценках	67
Приложение Б. Результаты при истинных кардинальностях	84

1. Введение

Реляционная СУБД хранит данные в виде таблиц, состоящих из строк и столбцов. Для формулирования запросов к данным используется декларативный язык SQL. С ростом объёма данных и сложности запросов увеличивается время, необходимое для их выполнения, и сокращение этого времени становится важным фактором эффективности СУБД.

Язык SQL Стандартным средством взаимодействия с реляционными СУБД является язык SQL (Structured Query Language) — декларативный язык запросов, в котором пользователь описывает *какие* данные необходимо получить, но не указывает *как* именно их извлечь. Типичный запрос на чтение данных имеет вид `SELECT ... FROM ... WHERE ...` и задаёт: список возвращаемых атрибутов, набор таблиц-источников и условия соединения между ними, а также фильтрующие условия на строки. Один и тот же результат может быть получен множеством различных способов исполнения; выбор конкретного способа — задача оптимизатора СУБД.

В процессе трансляции запрос превращается сначала в логическое представление в виде дерева, затем с помощью оптимизатора СУБД в физический план исполнения. Производительность СУБД напрямую зависит от качества построенного физического плана. Для создания "хорошего" плана нужно решить задачу выбора оптимального порядка соединений таблиц, которая является NP-полной [1]. В процессе работы оптимизатор определяет, какой тип соединения использовать и в каком порядке соединять таблицы; запрос преобразуется в набор планов.

Соединение таблиц – это операция, которая позволяет объединять данные из двух и более таблиц по определённому условию. Использование

соединений необходимо, когда информация распределена между несколькими таблицами. Каждый аргумент это таблица, либо результат соединения нескольких таблиц.

Отношение – либо единичная таблица, либо результат соединения нескольких таблиц.

Кардинальность отношения R — это число записей в R , обозначается $|R|$. Для базовой таблицы кардинальность известна из системного каталога. Для промежуточного результата $R_i \bowtie R_j$ кардинальность неизвестна заранее и оценивается оптимизатором на основе статистик; обозначим оценку $|R_i \bowtie R_j|$.

Селективность предиката соединения p — это доля строк декартова произведения $R_i \times R_j$, удовлетворяющих условию p :

$$\text{sel}(p) = \frac{|R_i \bowtie_p R_j|}{|R_i| \cdot |R_j|}, \quad 0 \leq \text{sel}(p) \leq 1.$$

Оптимизатор оценивает $\text{sel}(p)$ по статистикам, предполагая независимость столбцов.

Виды соединений таблиц *Внутреннее соединение* возвращает только строки, для которых условие связи выполняется в обоих аргументах. *Внешнее соединение* (левое, правое или полное) дополнительно возвращает строки одного или обоих аргументов, не имеющие совпадений, заполняя недостающие значения пустыми; в отличие от внутреннего, внешнее соединение не коммутативно и не ассоциативно уже на уровне семантики, что ограничивает множество допустимых порядков соединений.

Стадии обработки запроса Путь от текстового SQL-запроса до результата в PostgreSQL проходит через 4 этапа:

1. **Разбор** — лексический и синтаксический анализ текста запроса, построение дерева разбора.
2. **Семантический анализ** — разрешение имён таблиц и атрибутов по системному каталогу, проверка типов, построение дерева запроса.
3. **Применение правил переписывания** — раскрытие представлений, применение правил безопасности на уровне строк; семантика запроса сохраняется.
4. **Планирование** — единая стадия, включающая предварительные логические преобразования, извлечение базовых отношений $\mathcal{R}$ и предикатов соединения $\mathcal{P}$, построение графа соединений $G = (\mathcal{R}, \mathcal{E})$ и выбор порядка соединений с назначением физических алгоритмов каждому из них. Результат — дерево операторов, передаваемое исполнителю.

Стоимостная модель Оптимизатор СУБД использует функцию стоимости $\text{Cost} : \text{Resources}_n(T) \rightarrow \mathbb{R}_{\geq 0}$, которая сопоставляет прогнозам использования различных ресурсов (ввод-вывод, CPU, память, сетевые ресурсы) оценку ресурсозатрат плана T . Обозначим $\text{Cost}(\text{Resources}_n(T))$ как $\text{Cost}(T)$. Для этого используются статистики отношений (оценкам кардинальностей, селективностей предикатов, доступности индексов). В идеальном случае $\text{Cost}(T)$ монотонно коррелирует с фактическим временем исполнения плана T ; на практике эта корреляция нарушается из-за ошибок оценки кардинальностей промежуточных результатов [2].

Операция внутреннего соединения коммутативна и ассоциативна по результату: $(R_1 \bowtie R_2) \bowtie R_3$ и $R_1 \bowtie (R_2 \bowtie R_3)$ возвращают одно и то же множество строк. Однако *стоимость вычисления* может быть

разной: $\text{Cost}((R_1 \bowtie R_2) \bowtie R_3) \neq \text{Cost}(R_1 \bowtie (R_2 \bowtie R_3))$, поскольку кардинальности промежуточных результатов зависят от порядка соединений, и могут быть выбраны разные физические способы соединения отношений. Внешние соединения не ассоциативны и не коммутативны уже на уровне семантики: перестановка аргументов может изменить результат. Это дополнительно ограничивает множество допустимых порядков соединений.

Выбор, в какой последовательности нужно соединить таблицы, является задачей выбора порядка соединений. Различные планы исполнения для одного и того же запроса возвращают одинаковый результат, но их стоимости различны. Поэтому выбор оптимального плана позволяет сократить время отклика, минимизировать потребление ресурсов и эффективно обрабатывать большие массивы данных, значительно повышая удобство работы пользователя.

Запрос удобно представлять как граф соединений, где вершины — отношения, рёбра — условия соединений; для предикатов, затрагивающих более двух таблиц, используются гиперрёбра.

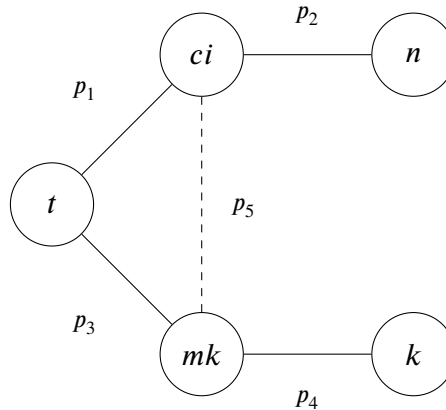
Пример запроса Рассмотрим SQL-запрос, который находит названия фильмов жанра «комедия», вышедших после 2010 года, вместе с именами актёров:

```

SELECT t.title, n.name
FROM   title      AS t,
       cast_info  AS ci,
       name       AS n,
       movie_keyword AS mk,
       keyword     AS k
WHERE  ci.movie_id = t.id
      AND ci.person_id = n.id
      AND mk.movie_id = t.id
      AND mk.keyword_id = k.id
      AND k.keyword = 'comedy'
      AND t.production_year > 2010;

```

После стадий разбора, семантического анализа и переписывания планировщик извлекает пять базовых отношений $\mathcal{R} = \{t, ci, n, mk, k\}$ и четыре предиката соединения. Поскольку два условия содержат общий атрибут $t.id$, планировщик выводит из них класс эквивалентности $\{ci.movie_id, mk.movie_id, t.id\}$ и порождает выводимый предикат $ci.movie_id = mk.movie_id$. Граф соединений G принимает вид:



где $p_1 : ci.movie_id = t.id$, $p_2 : ci.person_id = n.id$, $p_3 : mk.movie_id = t.id$,

$p_4: mk.keyword_id = k.id$, $p_5: ci.movie_id = mk.movie_id$ (пунктирное ребро — предикат, выведенный из класса эквивалентности). Фильтрующие условия $k.keyword = 'comedy'$ и $t.production_year > 2010$ применяются локально к таблицам k и t .

На стадии физической оптимизации планировщик перебирает порядки соединений. Один из возможных планов — левостороннее дерево $T_1 = (((k \bowtie mk) \bowtie t) \bowtie ci) \bowtie n$, которое начинает с малой таблицы k (после фильтра $keyword = 'comedy'$ остаётся одна строка), соединяет с mk по индексу, затем последовательно присоединяет t , ci , n . Промежуточные результаты растут постепенно, что минимизирует объём обработки. Альтернативный порядок, в котором сначала соединяются большие таблицы ci и t , породил бы промежуточный результат в миллионы строк уже на первом шаге, многократно увеличив время исполнения.

Комбинаторная сложность Если соединения выполнять бинарно, план задаётся полным двоичным деревом с n листьями (таблицами). Число различных форм таких деревьев — C_{n-1} , где C_k — число Каталана. С учётом перестановок таблиц по листьям число различных планов составляет $C_{n-1} \cdot n!$; асимптотически $C_{n-1} \cdot n! \sim \frac{4^{n-1} \cdot n!}{n^{3/2} \sqrt{\pi}}$. Экспоненциальный рост количества возможных планов.

Различные топологии графа (цепи, циклы, звёзды, плотные графы) приводят к разному числу допустимых планов, отличающемуся на порядки при росте числа таблиц.

На практике индустриальные СУБД комбинируют подходы. Для малого числа таблиц применяют динамическое программирование (перебор подмножеств с запоминанием лучших частичных планов). При росте количества таблиц переключаются на эвристики: жадные стратегии,

локальные улучшения, генетические алгоритмы и их простые сочетания. У такого переключения есть существенный недостаток: количество возможных планов коррелирует с числом таблиц, но линейная зависимость отсутствует.

В данной работе предложены и реализованы в ядре PostgreSQL несколько подходов, которые приводят к новому методу адаптивного планирования запросов — *Связывание якорных участков*.

2. Формальная постановка задачи

Пусть задан запрос, затрагивающий n базовых отношений $\mathcal{R} = \{R_1, \dots, R_n\}$, и множество предикатов соединения $\mathcal{P}$, определяющее граф соединений $G = (\mathcal{R}, \mathcal{E})$, где ребро $(R_i, R_j) \in \mathcal{E}$ существует тогда и только тогда, когда имеется предикат, связывающий R_i и R_j .

План исполнения запроса — это полное бинарное дерево T с n листьями, помеченными отношениями из $\mathcal{R}$, в котором каждый внутренний узел задаёт операцию соединения своих двух поддеревьев. Обозначим $\mathcal{T}_n$ — множество всех допустимых планов для данного запроса.

Алгоритм планирования Алгоритм планирования — это отображение $\mathcal{A}: (\mathcal{R}, \mathcal{P}, \Sigma, (D)) \rightarrow \mathcal{T}_n$, где Σ — множество доступных статистик (гистограммы, список самых частых значений, оценки уникальных значений для каждого атрибута), (D) — данные. Результат $T_{\mathcal{A}} = \mathcal{A}(\mathcal{R}, \mathcal{P}, \Sigma) \in \mathcal{T}_n$ — план исполнения. Работа алгоритма $\mathcal{A}$ характеризуется двумя величинами:

- $t_{\text{plan}}(\mathcal{A})$ — время работы алгоритма планирования (время перебора, оценки и выбора плана);
- $t_{\text{exec}}(\mathcal{A})$ — время исполнения выбранного плана $T_{\mathcal{A}}$ на данных.

Время исполнения зависит от качества выбранного плана: чем ближе $T_{\mathcal{A}}$ к истинному оптимуму по фактическому времени, тем меньше t_{exec} . Время планирования определяется алгоритмом $\mathcal{A}$: точные методы (полный DP-перебор) дают малое t_{exec} , но большое t_{plan} при росте n ; быстрые эвристики сокращают t_{plan} , но могут увеличить t_{exec} .

Целевая функция Суммарное время обработки запроса:

$$t_{\text{total}}(\mathcal{A}) = t_{\text{plan}}(\mathcal{A}) + t_{\text{exec}}(\mathcal{A}) \longrightarrow \min_{\mathcal{A}}.$$

Задача состоит в построении алгоритма планирования $\mathcal{A}^*$, минимизирующего t_{total} на заданном классе запросов $\mathcal{Q}$:

$$\mathcal{A}^* = \arg \min_{\mathcal{A}} \sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q).$$

3. Предшествующие работы

3.1. Задача выбора порядка соединений

В статье Ibaraki & Kameda [1] формализуется задача выбора оптимального порядка соединения n таблиц, при использовании вложенного цикла для сканирования таблиц и результатов соединений. Для заданного набора отношений и предикатов соединения выводится формула ожидаемого числа чтений страниц как функция от порядка соединяемых отношений, и требуется найти порядок, минимизирующий это значение. Авторы прямо отмечают, что задача нахождения минимального значения этой функции NP-полная.

Задача выбора оптимального вложенного порядка принимает на вход:

1. Статистики для формулы ожидаемого числа чтений страниц.
2. Описание запроса (граф соединений).
3. Пороговое значение B — существует ли такой порядок вложенности отношений, что число чтений страниц $\leq B$?

3.2. Принадлежность NP

Если мы угадали порядок отношений π , то значение функции числа чтений страниц для π вычисляется полиномиально по размеру входа. Значит, решение можно проверить за полиномиальное время, тогда задача принадлежит NP.

3.3. NP-полнота

В доказательстве NP-полноты авторы сводят к этой задаче классическую NP-полную задачу поиска полного графа [3]: по графу $G = (V, E)$ и числу k строится экземпляр запроса (в терминах отношений и условий соединения) и подбираются параметры стоимости так, что существует порядок с числом чтений страниц $\leq B$ тогда и только тогда, когда в G есть полный граф размера k .

3.4. Пространство поиска планов

Для заданного начального дерева T **пространство поиска планов** [4] — это множество всех различных порядков, которые можно получить, применяя к начальному плану последовательность из корректных преобразований (они сохраняют эквивалентность результатов, т.е семантическую корректность).

Виды корректных преобразований:

1. **Коммутативное:** $(R \bowtie_{RS} S) = (S \bowtie_{SR} R)$
2. **Ассоциативное:** $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = R \bowtie_{RS,RT} (S \bowtie_{ST} T)$
3. **Левассоциативное:** $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = (R \bowtie_{RT} T) \bowtie_{RS,ST} S$
4. **Правоассоциативное:** $R \bowtie_{RS,RT} (S \bowtie_{ST} T) = S \bowtie_{RS,ST} (R \bowtie_{RT} T)$

3.5. Жадный подход к планированию

В статье [5] предлагается эвристический способ построения оптимального порядка, путём ухода от экспоненциального перебора порядков соединений, строя порядок соединений жадно снизу-вверх.

Algorithm 1 $\text{GOO}(\{R_1, \dots, R_n\}, C(T_1, T_2))$

```
1: Input: set of relations to be joined
2: Input: join tree
3:  $Trees \leftarrow \{R_1, \dots, R_n\}$ 
4: while  $|Trees| \neq 1$  do
5:   find  $T_i, T_j \in Trees$  such that  $i \neq j$  and  $C(T_i, T_j)$  is minimal among all pairs
     of trees in  $Trees$ 
6:    $Trees \leftarrow Trees \setminus \{T_i\}$ 
7:    $Trees \leftarrow Trees \setminus \{T_j\}$ 
8:    $Trees \leftarrow Trees \cup \{T_i \bowtie T_j\}$ 
9: end while
10: return the (only) tree contained in  $Trees$ 
```

Преимущества:

1. Эвристика строит ветвистые деревья, что позволяет параллельно исполнять дерево плана.
2. Полиномиальность по времени планирования $O(n^3)$, где n — число начальных отношений, вместо экспоненциального времени у полного перебора.

Недостатки:

1. Нет гарантий оптимальности: локально лучший шаг может привести в глобально плохой порядок.
2. Нестабильность и зависимость от качества оценок кардинальностей/селективностей: если оценки ошибочны, жадный подход резко деградирует.

Прослеживается связь с работой [1]. Так как в общем виде задача выбора

оптимального порядка соединения отношений NP-полная, то планирование больших запросов требует больших вычислительных ресурсов, следовательно, нужны эвристики.

В современных СУБД в качестве базового подхода к планированию используется методы **динамического программирования**. Это методы точного решения задач оптимизации, которые разбивают исходную задачу на набор перекрывающихся подзадач и оптимальное решение строится из их оптимальных решений. Вместо повторного пересчёта результатов они их запоминают, снижая асимптотическую сложность по сравнению с полным перебором. Такие алгоритмы эффективны, когда пространство решений можно параметризовать состояниями (например, подмножествами, интервалами, путями в графе). Недостаток — рост памяти и экспоненциальное число состояний в худшем случае.

3.6. Динамическое программирование по размеру подпланов (DPsize)

Алгоритм DPsize[6] — вариант динамического программирования для построения оптимального ветвистого дерева соединений при условии, что граф соединений запроса связный. Алгоритм перечисляет планы в порядке возрастания размера подпланов.

DPsize поддерживает таблицу *BestPlan(S)*, сопоставляющую каждому множеству отношений *S* лучший (по минимальной стоимости) найденный план, и строит планы снизу-вверх:

Algorithm 2 DPsize($\{R_1, \dots, R_n\}, C(T_1, T_2)$)

```
1: Input: A set of relations  $R = \{R_1, \dots, R_n\}$  to be joined
2: Output: An optimal bushy join tree
3:  $B \leftarrow$  an empty DP table  $2^R \rightarrow$  join tree
4: for each  $R_i \in R$  do
5:    $B[\{R_i\}] \leftarrow R_i$ 
6: end for
7: for each  $1 < s \leq n$  ascending do
8:   for each  $S_1, S_2 \subset R$  such that  $|S_1| + |S_2| = s$  do
9:     if (not cross products  $\wedge \neg S_1$  connected to  $S_2$ )  $\vee (S_1 \cap S_2 \neq \emptyset)$  then
10:      continue
11:    end if
12:     $p_1 \leftarrow B[S_1], p_2 \leftarrow B[S_2]$ 
13:    if  $p_1 = \epsilon$  or  $p_2 = \epsilon$  then continue
14:    end if
15:     $P \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
16:    if  $B[S_1 \cup S_2] = \epsilon$  or  $C(B[S_1 \cup S_2]) > C(P)$  then
17:       $B[S_1 \cup S_2] \leftarrow P$ 
18:    end if
19:  end for
20: end for
```

Преимущества:

1. Структурированное перечисление снизу-вверх по размеру подпланов удобно для реализации.
2. Полиномиальность для простых топологий: для цепей и циклов число

внутренних проверок ($O(n^4)$).

Недостатки:

1. Экспоненциальность для сложных топологий: для звёзд и полных графов число внутренних проверок ($O(4^n)$).

3.7. Динамическое программирование по подмножествам (DPsub)

DPsub[6] описывается также как вариант динамического программирования для построения оптимального ветвистого дерева соединений для связного графа запроса. DPsub перебирает все непустые подмножества исходного множества таблиц и для каждого подмножества строит лучший план.

```

1: Input: A set of relations  $R = \{R_1, \dots, R_n\}$  to be joined
2: Output: An optimal bushy join tree
3:  $B \leftarrow$  an empty DP table  $2^R \rightarrow$  join tree
4: for each  $R_i \in R$  do
5:      $B[\{R_i\}] \leftarrow R_i$ 
6: end for
7: for each  $1 < i \leq 2^n - 1$  ascending do
8:      $S \leftarrow \{R_j \in R \mid (\lfloor i/2^{j-1} \rfloor \bmod 2) = 1\}$ 
9:     for each  $S_1, S_2 \subset S$  such that  $S_2 = S \setminus S_1$  do
10:        if (not cross products  $\wedge \neg S_1$  connected to  $S_2$ ) then
11:            continue
12:        end if
13:         $p_1 \leftarrow B[S_1], p_2 \leftarrow B[S_2]$ 
14:        if  $p_1 = \epsilon$  or  $p_2 = \epsilon$  then continue
15:        end if
16:         $P \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
17:        if  $B[S] = \epsilon$  or  $C(B[S]) > C(P)$  then
18:             $B[S] \leftarrow P$ 
19:        end if
20:    end for
21: end for

```

Преимущества:

1. Эффективней DPsize на плотных пространствах поиска(звезда/полный граф): в таких графах больше связных подмножеств и больше разбиений $S = S_1 \cup S_2$, проходящих проверки. Сложность ($O(3^n)$)

Недостатки:

1. Хуже для простых топологий (цепь/цикл): DPsub перебирает все подмножества S , но значительная часть из них несвязна. Сложность ($O(2^n)$) для цепей и ($O(n2^n)$) для циклов.

3.8. Динамическое программирование по парам связных подграфов (DPcsp)

Пусть дан граф соединений $G = (V, E)$. Введём понятие *csg-str-pair* (S_1, S_2) — в графе запроса выделим S_1, S_2 — связные непересекающиеся подграфы, такие что $S_1 \subseteq V, S_2 \subseteq V \setminus S_1$ и существует между ними ребро. (S_1, S_2) называются комплементарной парой связных подграфов.

Определим $\#csg$ — количество связных подграфов.

Определим $\#csp$ — количество комплементарных пар связанных подграфов.

В [7] предлагается алгоритм DPcsp для построения оптимального ветвистого дерева для связного графа запроса и получения нижней теоретической оценки сложности перебора, которая равна $\#csp$.

DPcsp поддерживает таблицу $BestPlan(S)$ и вместо перебора всех разбиений подмножеств рассматривает ровно комплементарные пары:

Algorithm 3 DPccp

```
1: Input: a connected query graph with relations  $R = \{R_0, \dots, R_{n-1}\}$ 
2: Output: an optimal bushy join tree
3: for all  $R_i \in R$  do
4:    $\text{BestPlan}(\{R_i\}) \leftarrow R_i$ 
5: end for
6: for all csg-cmp-pairs  $(S_1, S_2), S = S_1 \cup S_2$  do
7:    $p_1 \leftarrow \text{BestPlan}(S_1)$ 
8:    $p_2 \leftarrow \text{BestPlan}(S_2)$ 
9:    $\text{CurrPlan} \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
10:  if  $\text{cost}(\text{BestPlan}(S)) > \text{cost}(\text{CurrPlan})$  then
11:     $\text{BestPlan}(S) \leftarrow \text{CurrPlan}$ 
12:  end if
13:   $\text{CurrPlan} \leftarrow \text{CreateJoinTree}(p_2, p_1)$ 
14:  if  $\text{cost}(\text{BestPlan}(S)) > \text{cost}(\text{CurrPlan})$  then
15:     $\text{BestPlan}(S) \leftarrow \text{CurrPlan}$ 
16:  end if
17: end for
18: return  $\text{BestPlan}(\{R_0, \dots, R_{n-1}\})$ 
```

Преимущества:

1. Достижение нижней границы перебора: DPccp рассматривает ровно $\#ccp$.
2. Полиномиальность для цепей и циклов: $O(n^3)$.

Недостатки:

1. Экспоненциальность на сложных топологиях: $O(3^n)$ для полных графов

и $O(n2^n)$ для звёзд.

2. Сложность реализации: нужно эффективно перечислять csg-cmp-pairs без дубликатов и в порядке, корректном для DP, чтобы перед (S_1, S_2) уже были рассмотрены все непустые подмножества.

В статье алгоритм DPсср перечисляет csg в порядке, согласованном с DP, используя нумерацию, построенную поиском в ширину, вершин и запрет на включение вершин с меткой меньше стартовой, чтобы не порождать дубликаты. Для каждого S_1 процедура EnumerateCmp строит вторую часть пары S_2 только внутри дополнения $V \setminus S_1$: стартует с одиночных вершин из окрестности (S_1) и затем рекурсивно расширяет их, добавляя подмножества соседей уже построенного S_2 (то есть всегда растит связный компонент). Параметр исключения X выбирается так, чтобы S_2 не содержал вершин с меткой меньше любой вершины из S_1 (условие порядка $\min(S_1) < \min(S_2)$, где $\min(S)$ - минимальный номер вершины, входящей в S), что устраняет симметричные дубликаты $(S_1, S_2) / (S_2, S_1)$. Из-за старта из окрестности (S_1) и связного расширения гарантируется смежность S_2 к S_1 (есть ребро между компонентами), поэтому перечисляются ровно допустимые csg-cmp-пары .

3.9. DPhyp: динамическое программирование на гиперграфах

В другой статье, от тех же авторов [8] замечается, что DPсср эффективен для связных графов запросов, в котором все соединения внутренние и являются бинарными, т.е. каждое условие включает ровно две таблицы. Однако реальные запросы содержат *сложные предикаты*, связывающие сразу несколько отношений, и внешние соединения с ограничениями на

коммутативность и ассоциативность. DPhyr обобщает DPssr на эти случаи за счёт представления запроса в виде *гиперграфа*.

3.9.1. Гиперграф и csg-стр-пары

Запрос представим гиперграфом $H = (V, E)$, где V — таблицы, а гиперребро $e = (U, W)$ связывает две непересекающиеся группы $U, W \subseteq V$, $U \cap W = \emptyset$.

Комплементарная пара определяется аналогично, только вместо рёбер гиперёбра.

3.9.2. Идея перечисления

DPhyr перечисляет аналогично только сср. Вводится линейный порядок на отношениях $<$ и правило уникальности: рассматриваются только пары с $\min(S_1) < \min(S_2)$. Порядок нужен, чтобы симметричные дубликаты не порождались.

Компоненты строятся рекурсивным расширением через *окрестность* $N(S, X)$ — разрешённые узлы, достижимые из S по гиперрёбрам, исключая запрещённые X .

Пусть есть цепочка $R_1-R_2-R_3$, цепочка $R_4-R_5-R_6$ и гиперребро между $\{R_1, R_2, R_3\}$ и $\{R_4, R_5, R_6\}$. Тогда $(S_1, S_2) = (\{R_1, R_2, R_3\}, \{R_4, R_5, R_6\})$ — допустимая csg-стр-пара: обе стороны связны и соединяемы гиперребром. Характерная ситуация: при $S_1 = \{R_1, R_2, R_3\}$ окрестность даёт вход $\min(\{R_4, R_5, R_6\}) = R_4$, и процедура EnumerateCmpRec достраивает S_2 до полной тройки, после чего пара становится соединяемой.

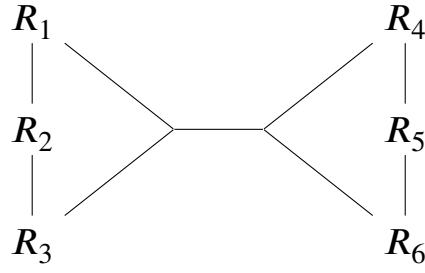


Рис. 3.1: Пример гиперграфа

3.9.3. DPhyp

Algorithm 4 DPhyp (перечисление csg-cmp-пар)

```

1: Input: Гиперграф запроса  $H = (V, E)$ , порядок  $<$  на  $V$ 
2: Output: an optimal bushy join tree.
3: global  $dp[\cdot]$   $\triangleright$  DP-таблица: множество отношений  $\rightarrow$  оптимальный план
4: function Solve
5:   for all  $v \in V$  do
6:      $dp[\{v\}] \leftarrow \text{Scan}(v)$ 
7:   end for
8:   for all  $v \in V$  in descending  $<$  do
9:      $\text{EmitCsg}(\{v\})$ 
10:     $B_v \leftarrow \{w \in V \mid w < v\} \cup \{v\}$ 
11:     $\text{EnumerateCsgRec}(\{v\}, B_v)$ 
12:   end for
13:   return  $dp[V]$ 
14: end function
15: function EnumerateCsgRec( $S_1, X$ )
16:    $Y \leftarrow \text{Neighborhood}(S_1, X)$ 
17:   for all  $N \subseteq Y$  with  $N \neq \emptyset$  do
18:     if  $\text{Connected}(S_1 \cup N)$  then
19:        $\text{EmitCsg}(S_1 \cup N)$ 
20:        $\text{EnumerateCsgRec}(S_1 \cup N, X \cup Y)$ 

```

Для внешних порядок перестановок ограничен семантикой, поэтому часть разбиений заведомо не корректна.

Идея статьи: закодировать такие ограничения в структуре гиперграфа, добавляя ограничивающие гиперребра, чтобы алгоритм DP перечислял только допустимые комбинации.

3.10. Генетический подход

В работе [9] предлагается генетический алгоритм для оптимизации запросов соединения как альтернатива классическому динамическому программированию типа DPsize. Стоимостная модель переводится в функцию приспособленности. Алгоритм пытается её минимизировать. Алгоритм поддерживает популяцию планов кандидатов, итеративно применяя селекцию, скрещивание и мутацию.

Ключевые идеи:

1. Кодирование планов:

(а) Хромосома — упорядоченный список генов, где каждый ген соответствует конкретному соединению из графа запроса, вместе с методом соединения и информацией о порядке.

2. Локальная селекция: отбор партнёров для скрещивания происходит внутри локального окружения хромосомы.

3. Операторы поиска:

(а) Мутации: случайная смена метода соединения или локальная перестановка соседних генов.

- (b) Перемещение: две перестановки и перенос случайного непрерывного фрагмента генов.

Преимущества

1. Масштабируемость: алгоритм не требует DP-таблиц экспоненциального размера, меньше потребление памяти.
2. Хорошая параллелизуемость: популяционная природа генетического подхода естественно переносится на параллельную архитектуру с небольшими коммуникационными затратами.

Недостатки

1. Нет гарантий оптимальности и стабильности: по мере роста размера запроса качество решений и устойчивость сходимости ухудшаются из-за резкого роста пространства поиска.
2. Чувствительность к параметрам: качество зависит от размера популяции, выбора операторов и схемы селекции; неудачные настройки дают деградацию.
3. Затраты на оценку приспособленности: нужно многократно вычислять стоимость планов для большого числа хромосом, что может доминировать во времени оптимизации.

3.11. Адаптивная оптимизация больших запросов соединения

В статье [10] предлагается адаптивный подход оптимизации порядка соединений, который выбирает стратегию планирования по сложности графа соединений и заданному бюджету перебора, чтобы для простых запросов

получить оптимум, а для сложных плавную и предсказуемую деградацию по времени оптимизации и качеству планов.

Ключевые идеи

1. Оценка сложности через число связных подграфов. Авторы считают #ссп графа соединений. Это число совпадает с размером полной DP-таблицы. Если число связных подграфов не превышает заданного порога, то DPhur считается быстрым, и запрос оптимизируется точно.
2. Линеаризация для средних запросов. Когда точный поиск становится слишком дорогим, вводится линеаризация пространства поиска: эвристически строится линейный порядок отношений, затем DPhur ограничивается только связными подцепочками этого порядка, вместо произвольных подмножеств. Это уменьшает размер DP-таблицы с $O(2^n)$ до $O(n^2)$ и снижает сложность поиска до порядка $O(n^3)$.
3. Жадный подход + локальная оптимизация больших поддеревьев для больших запросов. Для больших запросов строится начальный план жадным методом, после чего выполняется итеративное улучшение: выбранные поддеревья перепланируются более точным методом DP, но только до размера k .

Преимущества

1. Оптимальность для простых случаев.
2. Масштабирование до тысяч отношений: при росте размера запроса происходит переход к менее дорогим стадиям.
3. Плавная деградация качества.
4. Учитывается форма графа запроса: выбор алгоритма определяется не только числом начальных отношений, что точнее отражает реальную

сложность перечисления.

Недостатки

1. Линеаризация не применима при внешних соединениях и гиперрёбрах
2. Зависимость качества от линеаризации: выбранный линейный порядок ограничивает пространство планов, некоторые хорошие порядки соединений становятся недостижимыми.
3. Локальность улучшений в GOO-DP: перепланирование поддеревьев размера k не даёт полной свободы глобальных перестановок между различными поддеревьями, поэтому итог может оставаться далёким от оптимума.
4. Пороговые параметры являются эвристическими.

3.12. Архитектура DP планировщика PostgreSQL

Практическая часть данной работы выполнена на базе СУБД PostgreSQL — распространённой реляционной системы с открытым исходным кодом.

Планировщик PostgreSQL реализует алгоритм DPsize поуровнево. Пусть дан запрос с n отношениями. Планирование выполняется в цикле по уровню $s = 2, 3, \dots, n$, где s — число базовых отношений в подплане. На каждом уровне конвейер состоит из четырёх этапов:

1. **Перечисление пар** — процедура перебирает все разбиения целевого множества S ($|S| = s$) на два непересекающихся подмножества S_1 и S_2 , для которых уже найдены подпланы, и которые связаны хотя бы одним предикатом соединения. Перебор выполняется в фиксированном порядке: сначала левосторонние планы, затем ветвистые.

2. **Построение путей** — для каждой допустимой пары (S_1, S_2) вызывается процедура, которая конструирует конкретные физические пути исполнения соединения. Эта процедура является самой дорогой частью планирования: по данным профилирования, на неё приходится свыше 90% времени t_{plan} .
3. **Инкрементальный отбор путей** — каждый построенный путь p_{new} сравнивается со всеми ранее сохранёнными путями p_{old} для того же множества S по трём независимым критериям: стоимость, порядок сортировки и параметризация. Сравнение стоимостей выполняется с допуском $\varepsilon = 1\%$: путь считается дешевле только при различии более ε по всем координатам; иначе стоимости нечётко равны. Путь p_{new} отбрасывается, если существует p_{old} , доминирующий по всем трём критериям; симметрично p_{old} удаляется, если его доминирует p_{new} . При полной эквивалентности сохраняется путь, поступивший *раньше*.
4. **Финализация уровня** — после перебора всех планов подмножеств для каждого вновь созданного множества S выбирается путь с минимальной оценочной стоимостью из числа оставшихся. Указатель на этот путь фиксируется и используется при построении подпланов на уровне $s + 1$.

Порядок перебора как скрытый параметр Пусть для множества S рассмотрены два плана: $S = S_1^{(a)} \cup S_2^{(a)}$ (левостороннее, перебрано первым) и $S = S_1^{(b)} \cup S_2^{(b)}$ (ветвистое, перебрано позже). Обозначим построенные пути p_a и p_b соответственно. Если $\text{Cost}(p_a) \approx \text{Cost}(p_b)$ с точностью ε , то p_a остаётся в множестве сохранённых, а p_b отклоняется. При обратном порядке перебора остался бы p_b .

При точных оценках кардинальностей этот эффект незначителен,

однако при ошибочных оценках стоимости конкурирующих путей часто попадают в ε -окрестность, и порядок перебора фактически предопределяет результат. Поскольку левосторонние планы обрабатываются первыми на каждом уровне, планировщик PostgreSQL имеет систематическое структурное смещение в пользу левосторонних планов.

Число вызовов процедуры построения путей Количество пар (S_1, S_2) , для которых вызывается процедура построения путей, определяет время планирования t_{plan} . На уровне s число допустимых разбиений зависит от топологии графа соединений G . Для полного графа из n вершин на уровне s рассматриваются $C_n^k \cdot (2^s - 2)/2$ разбиений, с максимумом при $s \approx n/2$. Распределение числа вызовов по уровням имеет характерную форму: малое число на крайних уровнях ($s = 2$ и $s = n$) и максимум на средних уровнях, где сосредоточена основная вычислительная нагрузка планирования.

3.13. Оценки в модели стоимости

В статье [2] авторы исследуют, насколько хорошо оптимизаторы промышленных СУБД справляются с выбором порядка соединений на реальных данных. Используется Join Order Benchmark (JOB) на базе данных IMDb. Модифицированный PostgreSQL получает доступ к *истинным* кардинальностям промежуточных результатов, что позволяет отделить влияние ошибок оценок от других компонентов оптимизатора.

Сбор и представление статистик Оценки селективностей и кардинальностей строятся по статистикам. Для каждого столбца каждой таблицы вычисляются три структуры: число различных значений n_{distinct} (для оценки селективности равенства $\text{sel}(A = c)$ в предположении равномерного

распределения); список из d наиболее частых значений с их частотами; гистограмма равновысоких корзин.

Увеличение d уменьшает ошибки оценок для одиночных предикатов на одном столбце, однако *не решает* проблему оценки совместных распределений нескольких столбцов из разных таблиц. Статистики собираются независимо для каждого столбца, и оценка селективности соединения строится в предположении независимости:

$$\text{sel}(R_i.A = R_j.B) \approx \frac{1}{\max(n_{\text{distinct}}(A), n_{\text{distinct}}(B))}.$$

При наличии корреляций между атрибутами разных таблиц это предположение нарушается, и ошибки возрастают мультипликативно по цепочке соединений.

Ошибки оценок кардинальностей Стоимостная модель оптимизатора вычисляет $\text{Cost}(T)$ по оценкам кардинальностей промежуточных результатов $|S|$ для каждого подплана S в дереве T . Авторы показывают, что для запросов с k соединениями ошибки кардинальности растут мультипликативно: если оценка каждого единичного предиката ошибается в q раз, то после k соединений накопленная ошибка может достигать q^k . На JOB для запросов с 5–17 таблицами зафиксированы ошибки в 10^2 – 10^6 раз относительно истинных кардинальностей промежуточных результатов. При подстановке истинных кардинальностей планы оптимизатора улучшаются на порядки по времени исполнения, тогда как изменение параметров самой стоимостной функции влияет значительно слабее.

Сравнение подпланов Стоимость $\text{Cost}(T)$ задумана как величина, монотонно коррелирующая с фактическим временем исполнения t_{exec} . Однако

при ошибочных оценках кардинальностей эта корреляция нарушается: два плана T_1 и T_2 могут иметь близкие оценочные стоимости $\text{Cost}(T_1) \approx \text{Cost}(T_2)$, но кратно отличающееся фактическое время исполнения.

Leis и др. [2] показали на JOB, что при истинных кардинальностях медианная ошибка стоимостной модели PostgreSQL составляет 38%, а замена сложной модели на простую меняет среднее время исполнения на 34–41%. При этом замена оценок кардинальностей на истинные сокращает время исполнения в несколько раз. Амплитуда шума, вносимого стоимостной моделью поверх кардинальности, на сильно меньше амплитуды ошибок оценки кардинальности (10^2 – 10^6) и не способна изменить относительный порядок планов, разделённых по кардинальности.

Отсюда следует практическое правило сравнения подпланов. Для подпланов малого размера ошибки оценок кардинальностей ещё не успевают накопиться и сравнение по стоимости остаётся надёжным. Для подпланов большего размера сравнение по кардинальности оказывается устойчивее. Это правило обосновывает переключение критерия сравнения в подходе GOO_{comb} (описан в ходе исследования).

4. Ход исследования

4.1. Окружение

Все эксперименты выполнены на одной машине с фиксированной конфигурацией, чтобы обеспечить воспроизводимость и сопоставимость результатов.

Аппаратное обеспечение.

- Процессор: Intel Core Ultra 7 258V (8 ядер).
- Оперативная память: 32 ГБ.
- Операционная система: Linux Ubuntu 25.04.

СУБД. PostgreSQL версии 18.2

Бенчмарк. В данной работе в качестве Q используются два индустриальных бенчмарка: Join Order Benchmark (JOB) [2] — 113 запросов с 5–17 соединениями к базе IMDb — и JOB-Complex [11] — 30 запросов к той же базе с реалистичными свойствами: соединения по строковым атрибутам, не первичные, внешние ключи, сложные фильтры с множественными значениями. IMDb (Internet Movie Database) содержит информацию о фильмах, актёрах, ключевых словах и связанных сущностях. Запросы JOB содержат от 5 до 17 соединений, коррелированные фильтры и разнообразные топологии графов соединений, что делает бенчмарк стандартным инструментом для оценки качества планировщиков соединений.

Эти свойства приводят к значительным ошибкам оценки кардинальностей: PostgreSQL не имеет совместных статистик по строковым столбцам разных таблиц, и предположение независимости нарушается на порядки. Авторы [11] показали, что PostgreSQL выбирает планы в

11,1 раза медленнее оптимальных на JOB-Complex (против 2 раз на JOB). Использование JOB-Complex позволяет оценить устойчивость предложенных подходов в условиях сильно ошибочных оценок кардинальностей.

Полученные в работе результаты относятся к конкретным $\mathcal{Q}$ (JOB и JOB-Complex) и не претендуют на существование универсального $\mathcal{A}^*$, минимизирующего t_{total} для произвольного класса запросов.

Горячие данные. Все данные были загружены в буферный кэш СУБД. Это исключает влияние дискового ввода-вывода на измерения.

Конфигурация СУБД. Параметры PostgreSQL выбраны так, чтобы устранить влияние побочных эффектов (дисковый ввод-вывод, нехватка памяти для операторов) на время исполнения запросов, изолируя эффект от выбора порядка соединений:

- `shared_buffers` = 12 GB — размер буферного кэша СУБД. Достаточен для размещения всей базы IMDb в оперативной памяти.
- `work_mem` = 8 GB — объём памяти, выделяемый каждой операции сортировки или хеширования.
- `effective_cache_size` = 8 GB — подсказка оптимизатору об объёме доступного кэша операционной системы.
- `max_parallel_workers` = 8, `max_parallel_workers_per_gather` = 8 — параметры параллелизма.

Метрики экспериментальной оценки. Прямое сравнение алгоритмов по абсолютному значению $\sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q)$ чувствительно к нескольким запросам с большим t_{exec} , которые могут доминировать в сумме и маскировать поведение на остальных. Поэтому в работе используются три согласованные метрики, каждая из которых отражает свой аспект близости алгоритма $\mathcal{A}$ к

решению задачи $\mathcal{A}^* = \arg \min_{\mathcal{A}} \sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q)$:

- **Нормализованное суммарное время** — отношение $\sum_q t_{\text{total}}(\mathcal{A}, q) / \sum_q t_{\text{total}}(\mathcal{A}_{\text{DP}}, q)$, где $\mathcal{A}_{\text{DP}}$ — стандартный DPsize PostgreSQL. Значение < 1 означает, что $\mathcal{A}$ ближе к $\mathcal{A}^*$ по целевой функции, чем базовый алгоритм.
- **Нормализованные отношения по компонентам** — те же отношения отдельно для t_{plan} и t_{exec} . Позволяют различать улучшения за счёт ускорения планирования и за счёт выбора лучшего плана.
- **Число, где лучше** — количество запросов $q \in \mathcal{Q}$, на которых $t_{\text{total}}(\mathcal{A}, q) < t_{\text{total}}(\mathcal{A}_{\text{DP}}, q)$. Характеризует стабильность алгоритма: суммарное отношение может быть малым из-за нескольких крупных выигрышей при общей нестабильности.

4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения

Мотивация. Классическая реализация использует оценочную стоимость в качестве критерия. Однако, как показано в секции «Оценки в модели стоимости», при увеличении числа соединений ошибки оценок кардинальностей каскадируются и стоимость теряет способность различать планы по истинному качеству. Возникает вопрос: можно ли подобрать критерий сравнения, более устойчивый к ошибкам оценок?

Реализованы и протестированы три варианта GOO, различающиеся только критерием выбора лучшей пары на каждом шаге.

Вариант 1: GOO по стоимости (GOO_{cost}). Критерий — оценочная

стоимость результата соединения:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} \text{Cost}(R_i \bowtie R_j).$$

Это классическая реализация GOO, в которой на каждом шаге выбирается пара с минимальной стоимостью по модели СУБД.

Вариант 2: GOO по кардинальности (GOO_{card}). Критерий — оценка объёма результата соединения:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} (|\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j)),$$

где $|\widehat{R_i \bowtie R_j}|$ — оценочная кардинальность, а $w(R_i \bowtie R_j)$ — средняя ширина строки результата (в байтах). Произведение $|\widehat{R_i \bowtie R_j}| \cdot w$ оценивает объём данных промежуточного результата и является более грубым, но более устойчивым критерием, чем стоимость на подпланах большого размера.

Вариант 3: GOO с комбинированным критерием (GOO_{comb}). Критерий адаптивно переключается в зависимости от размера аргументов соединения. Пусть $|S|$ — число базовых отношений в подплане S . Пара (R_i, R_j) называется *простой*, если $|R_i| \leq 2$ и $|R_j| \leq 2$. Критерий выбора:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} \begin{cases} \text{Cost}(R_i \bowtie R_j), & \text{если пара простая,} \\ |\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j), & \text{иначе.} \end{cases}$$

Результаты. Все три варианта протестированы на 113 запросах JOB. Суммарные отношения к DPsize:

	t_{plan}	t_{exec}	t_{total}	#лучше
GOO _{cost}	0,85	1,44	1,22	55/113
GOO _{card}	0,86	1,62	1,34	64/113
GOO _{comb}	0,81	1,19	1,05	80/113

Все три варианта сокращают t_{plan} , но проигрывают по t_{exec} . GOO_{comb} - наибольший суммарный выигрыш.

Случай, когда жадные подходы превосходят DP. При значительных ошибках оценки кардинальностей DP может выбрать план, оптимальный по ошибочным оценкам, но неудачный по фактическому времени. Жадный алгоритм перебирает меньше вариантов, но его выбор на каждом шаге менее подвержен каскадному накоплению ошибок, поскольку опирается на локальное сравнение пар, а не на глобальную стоимость всего дерева.

Пример: запрос 6d ($t_{\text{total}}^{\text{DP}} = 2272$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 212$ мс, улучшение в 10,7 раза). Запрос содержит фильтр по таблице n (name): name LIKE '%Downey%Robert%', оставляющий 2 строки.

DP строит левосторонний план $k \rightarrow mk \rightarrow t \rightarrow ci \rightarrow n$, в котором таблица n стоит последней. Стоимостная модель недооценивает селективность шаблонного предиката LIKE и оценивает промежуточный результат ci как небольшой. В действительности до соединения с n кардинальность достигает 785 477 строк, полученных сканированием по индексу.

GOO_{comb} на первом шаге соединяет n (2 строки после фильтра) с ci через индексное сканирование (243 строки), поскольку эта пара имеет минимальную оценку кардинальности среди всех пар. Далее результат соединяется хеш-соединением с веткой $k \rightarrow mk \rightarrow t$ (14 165 строк). Промежуточный результат после первого шага — 243 строки вместо 785 477. Время исполнения: 117 мс вместо 2163 мс.

Данный пример иллюстрирует общий механизм: когда одна из таблиц имеет высокоселективный фильтр, который стоимостная модель не в состоянии точно оценить, жадный выбор по кардинальности помещает её в начало цепочки соединений, тогда как DP откладывает её на конец.

Случаи деградации жадных подходов. Жадный алгоритм принимает на каждом шаге необратимое решение на основании локальной информации. Если выбранное соединение создаёт промежуточный результат, не позволяющий эффективно фильтровать на последующих шагах, деградация неизбежна.

Пример: запрос 17c ($t_{\text{total}}^{\text{DP}} = 3086$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 10\,112$ мс, деградация в 3,3 раза). На ранних шагах GOO_{comb} соединяет $k \rightarrow mk \rightarrow t \rightarrow mc \rightarrow cn$: каждая пара имеет малую оценку кардинальности, и на каждом шаге выбор выглядит оптимальным. Однако построенная ветвь не содержит таблицу n (name), имеющую высокоселективный фильтр name LIKE '%B%'. Когда на последних шагах присоединяются ci (52,5 строки на итерацию, 148 552 итераций) и затем n (7 796 926 итераций), фильтрация по n применяется поздно, и промежуточные результаты оказываются на порядок больше, чем в плане DP.

DP находит порядок, в котором ci соединяется с n рано (ещё до mc и cn), что позволяет фильтру по имени сократить промежуточный результат до 250 строк. Жадный алгоритм не способен предвидеть этот эффект, поскольку на момент принятия решения таблица n не связана предикатом с текущим промежуточным результатом.

Результаты на JOB-Complex. На 30 запросах JOB-Complex картина меняется радикально:

	t_{plan}	t_{exec}	t_{total}	#лучше
GOO _{cost}	0,81	1,92	1,85	9/30
GOO _{card}	0,86	0,06	0,11	17/30
GOO _{comb}	0,85	0,13	0,17	16/30

GOO_{card} показывает улучшение t_{total} в 9 раз относительно DPsize — результат, диаметрально противоположный JOB, где GOO_{card} проигрывал ($t_{\text{total}} = 1,34$). GOO_{comb} также улучшает t_{total} в 5,9 раз. Напротив, GOO_{cost} деградирует (1,85), подтверждая, что на JOB-Complex стоимостная модель сильно ошибается.

JOB-Complex содержит соединения по строковым столбцам без первичных/внешних ключей, для которых ошибки оценки кардинальностей достигают 10^5 – 10^8 . При таких ошибках DP выбирает планы со вложенными циклами, оптимальные по стоимости, но долгие по фактическому времени. Например, запрос 12: DPsize выбирает вложенный цикл с $2,6 \cdot 10^9$ итераций ($t_{\text{exec}} = 539$ с), тогда как GOO_{card} находит хеш-соединение ($t_{\text{exec}} = 640$ мс, улучшение в 840 раз). GOO_{card}, поскольку не использует стоимостную модель и не может выбрать вложенный цикл на основании ошибочно заниженной оценки кардинальности.

4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии

Мотивация. Как отмечается в [6,10], время работы алгоритмов перечисления подпланов существенно зависит от топологии графа соединений. Для цепи из n вершин DPсср перебирает $O(n^3)$ пар связных подграфов, для звезды — $O(n \cdot 2^n)$, для полного графа — $O(3^n)$. Возникает

естественная идея: если граф запроса можно разложить на подграфы с известной топологией, каждый можно спланировать специализированным алгоритмом с меньшей сложностью, чем общий DP для всего графа.

Описание подхода. Граф соединений итеративно разбивается на топологические фрагменты (плотные подграфы, звёзды, циклы, цепи) — в фиксированном порядке приоритетов. Каждый фрагмент планируется специализированным алгоритмом: цепи и циклы — интервальным DP, звёзды — планированием лучей с жадной сборкой к центру, плотные подграфы — DPsub. Результат планирования фрагмента становится вершиной нового графа; процесс повторяется до схождения к одной вершине. Финальная сборка компонент выполняется жадным алгоритмом GOO_{comb} .

Результаты. Подход протестирован на всех 113 запросах JOB и 30 запросах JOB-Complex.

Бенчмарк	t_{plan}	t_{exec}	t_{total}	#лучше
JOB	0,83	3,55	2,55	32/113
JOB-Complex	0,82	3,12	2,99	12/30

Экономия t_{plan} в 17% достигается за счёт меньшего числа вызовов процедуры построения путей в каждом фрагменте, но деградация t_{exec} в 3,1–3,6 раза её полностью перекрывает. Из 113 запросов JOB лишь 32 показали улучшение t_{total} ; 81 запрос деградировал, причём 10 запросов — более чем в 5 раз.

Причина деградации: потеря параметризованных путей через центральные вершины. В нормализованных реляционных схемах существуют центральные таблицы, к которым через внешние ключи присоединяются многие другие. В схеме IMDb такой вершиной является таблица t (title): к ней присоединяются ci , mc , mk , mi , mi_idx , cc . При

декомпозиции центральная попадает в один фрагмент, а соседние таблицы из других фрагментов теряют возможность использовать индексное сканирование по первичному ключу таблицы. Вместо параметризованного индексного сканирования оптимизатор вынужден использовать хеш-соединение без индекса.

Пример: запрос 10а. Запрос соединяет 7 таблиц, включая центральную вершину t и большие таблицы ci , mc (кардинальности ~ 36 млн, $\sim 2,5$ млн, $\sim 2,6$ млн). Стандартный DPsize строит левосторонний план с промежуточными кардинальностями $681 \rightarrow 4395 \rightarrow 2270 \rightarrow 76 \rightarrow 56 \rightarrow 52 \rightarrow 52$, используя индексное сканирование t по первичному ключу при каждом присоединении ($t_{\text{exec}} = 176$ мс).

Алгоритм декомпозиции относит t в один фрагмент с mc , а ci — в другой фрагмент с rt и chn . После планирования фрагментов новая вершина $\{mc, t\}$ содержит 207 410 строк (хеш-соединение двух полностью сканированных таблиц), а вершина $\{rt, ci, chn\}$ — 25 523 строки. На этапе сборки эти две новые вершины соединяются хеш-соединением по $t.id = ci.movie_id$, обрабатывая $207\,410 \times 25\,523$ кандидатов вместо 76 строк параметризованного индексного сканирования в плане DPsize. t_{exec} растёт с 176 мс до 2,66 секунд (деградация в 15,1 раза). Аналогичная потеря параметризованных путей через t наблюдается в 78 из 81 деградировавших запросов.

Вывод. Подход с декомпозицией на топологии бесперспективен для схем с выраженными центральными вершинами, характерными для нормализованных реляционных баз данных. Корень проблемы — разрушение параметризованных индексных путей при изоляции центральных вершин в один фрагмент. Это наблюдение мотивирует *безопасную* стратегию сокращения пространства поиска, в которой центральные не могут быть

изолированы от своих соседей.

4.4. Подход 3. DPhyr (реализация с открытым исходным кодом)

Мотивация. Алгоритм DPhyr [8] достигает нижней теоретической границы перебора среди DP-алгоритмов для произвольных графов соединений: он перечисляет *только* пары связных подграфов (csg-сmp-pairs), тогда как DPsize перебирает все подмножества, проверяя связность апостериори. Для разреженных графов (цепи, звёзды) это может дать экспоненциальное сокращение числа рассматриваемых пар.

Помимо теоретической границы перебора, DPhyr меняет *порядок*, в котором кандидаты поступают в процедуру построения путей: DPsize обрабатывает подпланы по возрастанию размера, а DPhyr — обходом по соседям гиперграфа. Поскольку результат планирования в PostgreSQL зависит от порядка перебора (см. «Архитектура планировщика PostgreSQL»), эксперимент позволяет одновременно проверить две гипотезы: (1) приводит ли достижение нижней теоретической границы перебора к улучшению t_{plan} и (2) может ли альтернативный порядок обхода систематически улучшить t_{exec} .

Описание. Использована реализация DPhyr с открытым исходным кодом, интегрированная в PostgreSQL в виде расширения. Алгоритм строит гиперграф соединений и перечисляет пары связных подграфов (S_1, S_2) обходом по соседям, гарантируя, что каждая перечисленная пара связна и соединена хотя бы одним предикатом. Для каждой пары вызывается процедура построения путей, аналогичная стандартному DPsize.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan} : 1,015 (деградация 1,5%),
- t_{exec} : 1,131 (деградация 13,1%),
- t_{total} : 1,088 (деградация 8,8%).

Алгоритм, достигающий нижней теоретической границы перебора, показал результат *хуже* стандартного DPhur по всем трём метрикам.

Результаты на JOB-Complex. На 30 запросах JOB-Complex DPhur также уступает DPhur: $t_{\text{plan}} = 1,06$, $t_{\text{exec}} = 1,10$, $t_{\text{total}} = 1,10$, #лучше 8 из 30. При этом на отдельных запросах с плохими планами DPhur (запросы 5, 11, 12, 20, где DPhur выбирает планы с вложенными циклами с 10^9 итераций) DPhur уступает им в 1,5–2 раза, тогда как GOO_{comb} находит на порядок более быстрый план. Это показывает, что изменённый порядок перечисления DPhur *не устраняет* сильные ошибки стоимостной модели на JOB-Complex, а меняет их распределение.

Анализ причин.

1. *Теоретическое преимущество не реализуется на практике.* Свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на само перечисление пар. DPhur и DPhur вызывают эту процедуру для одних и тех же допустимых пар, различаясь лишь порядком вызовов.

2. *Изменённый порядок перечисления ухудшает t_{exec} .* Это главная причина деградации. DPhur перечисляет пары обходом по соседям гиперграфа, а DPhur — в порядке возрастания размера подмножества. Как показано в секции «Архитектура планировщика PostgreSQL», при ошибках оценок кардинальностей стоимости конкурирующих путей попадают в ε -окрестность, и путь, поступивший в процедуру инкрементального отбора раньше, получает структурное преимущество. DPhur на каждом

уровне сначала обрабатывает левосторонние планы, где на каждом уровне присоединяются единичны отношения, для которых стоимостная модель PostgreSQL наиболее точна. DPhyp не имеет этого свойства: порядок перечисления определяется структурой гиперграфа и может отдавать приоритет ветвистым планам с менее надёжными оценками.

Вывод. Эксперимент с DPhyp показывает, что замена алгоритма перечисления при сохранении пространства поиска и процедуры построения путей не даёт систематического улучшения t_{total} . Причина — механизм ε -зависимости от порядка перебора: при ошибочных оценках кардинальностей стоимости конкурирующих путей попадают в ε -окрестность, и изменение порядка перечисления может как улучшить, так и ухудшить отдельные запросы, без устойчивого преимущества. Более того, если новый порядок систематически смещает приоритет в сторону планов с менее надёжными оценками, результат становится хуже DPsize.

Отсюда следуют два структурных ограничения для t_{total} , не устранимые заменой алгоритма перечисления:

- t_{exec} сильно зависит от качества оценок кардинальностей и порядком поступления путей в процедуру инкрементального отбора, а не алгоритмом обнаружения пар.
- t_{plan} определяется числом вызовов процедуры построения путей, которое при неизменном пространстве поиска совпадает у DPsize и DPhyp с точностью до фильтрации несвязных пар.

Для улучшения t_{total} необходимо либо сократить пространство поиска (фиксируя соединения с предсказуемыми оценками до основного DP), либо целенаправленно упорядочить перебор так, чтобы приоритет получали планы с надёжными оценками. Первое направление реализовано в подходе со

связыванием якорных участков (подход 5), второе — в модификации DPhyr, описанной далее (подход 4).

Следует подчеркнуть, что результат характеризует данную реализацию DPhyr, а не алгоритм в целом. Полноценная реализация с корректным построением обобщённых гиперрёбер и обнаружением конфликтов может показать иные результаты. Тем не менее пункты 1 (теоретическое преимущество не реализуется при $n \leq 17$) и 2 (зависимость от порядка перечисления) являются свойствами архитектуры PostgreSQL, а не конкретной реализации DPhyr.

4.5. Подход 4. Модифицированный DPhyr (DPhyrMod)

Мотивация. Эксперимент с DPhyr (подход 5) выявил три конкретные причины деградации: нарушение поуровневого инварианта PostgreSQL, невыгодный порядок обработки кандидатов и избыточные вызовы процедуры построения путей на промежуточных уровнях. Модифицированный DPhyr адресует каждую из этих причин отдельной модификацией, сохраняя оригинальный алгоритм перечисления csg-cmp-пар без изменений.

Модификация 1. Поуровневая материализация. Стандартный DPhyr строит отношения в порядке обхода гиперграфа, произвольно чередуя подпланы разных уровней. Однако архитектура планировщика PostgreSQL опирается на инвариант: все отношения уровня k должны быть полностью построены и финализированы до начала построения отношений уровня $k + 1$. Нарушение этого инварианта приводит к неполному распространению классов эквивалентности и пропуску путей.

Модификация разделяет работу DPhyr на две фазы:

1. **Перечисление.** Оригинальный алгоритм DPhyr перечисляет все csg-str-pairs, но не вызывает процедуру построения путей. Вместо этого для каждой пары (S_1, S_2) записывается список кандидатов в соответствующий узел DP-таблицы и вычисляется облегчённая оценка кардинальности $|\widehat{S_1 \bowtie S_2}|$.
2. **Материализация.** Узлы DP-таблицы группируются по уровню (числу базовых отношений) и обрабатываются строго последовательно: сначала все узлы уровня 2, затем уровня 3, и т. д. Для каждого узла вызывается процедура построения путей по всем его кандидатам, после чего выполняется финализация. Это восстанавливает поуровневый инвариант PostgreSQL.

Модификация 2. Приоритет левосторонние планов. Как показано в секции «Архитектура планировщика PostgreSQL», стандартный DPsize на каждом уровне обрабатывает левосторонние подпланы ($|S_1| = s - 1, |S_2| = 1$) до ветвистых. Это обеспечивает приоритет параметризованных путей (вложенный цикл с индексным сканированием) в процедуре инкрементального отбора. В DPhyr порядок кандидатов определяется обходом гиперграфа и не имеет этого свойства.

Модификация вводит два уровня сортировки:

1. **Между узлами одного уровня.** Узлы уровня k группируются по значению μ — минимальному размеру дочернего подплана среди всех кандидатов узла. Узлы с $\mu = 1$ (левосторонние) обрабатываются первыми, затем $\mu = 2$, и т. д.
2. **Внутри списка кандидатов одного узла.** Пары кандидатов сортируются по $\min(|S_1|, |S_2|)$ по возрастанию, так что левосторонние

планы узла поступают в процедуру построения путей раньше ветвистых. Обе сортировки воспроизводят порядок обработки стандартного DPsize в контексте DPhur-перечисления.

Модификация 3. Отсечение на промежуточных уровнях по кардинальности. Распределение числа узлов DP-таблицы по уровням имеет характерную форму нормального распределения с максимумом на средних уровнях ($s \approx n/2$). Для запроса с $n = 11$ таблицами на уровнях 5–8 сосредоточено $\sim 86\%$ всех узлов. Основная вычислительная нагрузка планирования приходится именно на эти уровни.

На каждом уровне из области горба ($\lceil n/3 \rceil + 1 \leq s \leq n - \lfloor n/3 \rfloor$) выполняется отсечение узлов с высокой оценкой кардинальности:

1. Вычисляется $\min_s |\widehat{S}|$ среди всех узлов уровня s .
2. Узлы с $|\widehat{S}| > \alpha \cdot \min_s |\widehat{S}|$ отсекаются (в реализации $\alpha = 300$).
3. Дополнительно отсекаются узлы с $|\widehat{S}| > C_{\text{out}}^{\text{GOO}}$, где $C_{\text{out}}^{\text{GOO}}$ — сумма кардинальностей промежуточных результатов жадного плана, вычисленная облегчённым GOO по оценкам $|\widehat{S}|$ из фазы перечисления. Если кардинальность одного промежуточного результата превышает суммарный объём всех промежуточных результатов жадного плана, такой подплан заведомо неоптимален.
4. **Гарантия покрытия.** После отсечения проверяется, что каждое базовое отношение $R_i \in \mathcal{R}$ входит хотя бы в один выживший узел уровня s . Если отношение потеряно, восстанавливается узел с минимальной $|\widehat{S}|$ среди отсечённых, содержащий это отношение. Это предотвращает отсечение всех путей через критически важные базовые отношения.

На крайних уровнях ($s \leq \lceil n/3 \rceil$ и $s > n - \lfloor n/3 \rfloor$) отсечение не выполняется:

число узлов мало, стоимостная модель ещё точна, а разнообразие путей критически важно для качества финального плана.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan} : 0,966 (экономия 3,4%),
- t_{exec} : 0,967 (улучшение 3,3%),
- t_{total} : 0,967 (улучшение 3,3%).

Из 113 запросов 98 показали улучшение t_{total} , что значительно лучше, чем исходный DPhyp (78 из 113).

Абсолютные суммарные времена (по 113 запросам):

- DPsize: $t_{\text{plan}} = 52,3$ с, $t_{\text{exec}} = 88,8$ с, $t_{\text{total}} = 141,2$ с.
- DPhypMod: $t_{\text{plan}} = 50,5$ с, $t_{\text{exec}} = 85,9$ с, $t_{\text{total}} = 136,5$ с.
- DPhyp: $t_{\text{plan}} = 53,1$ с, $t_{\text{exec}} = 100,5$ с, $t_{\text{total}} = 153,6$ с.

Анализ: почему улучшение ограничено 3,3%.

1. *Пространство поиска совпадает с DPsize.* Все три модификации влияют на *порядок обработки* и *количество* узлов DP-таблицы, но не расширяют и не сужают множество рассматриваемых разбиений по сравнению с DPsize. Поуровневая материализация и сортировка кандидатов восстанавливают инварианты стандартного планировщика — но не превосходят их. DPhypMod *приближается* к DPsize по качеству планов, а не улучшает его.

2. *Отсечение экономит мало.* Запросы JOB содержат от 5 до 17 таблиц. При таких размерах число узлов DP-таблицы составляет от десятков до нескольких тысяч, а горб распределения — область, где отсечение действует — содержит от десятков до нескольких сотен узлов. Отсечение по кардинальности с консервативным порогом $\alpha = 300$ и гарантией покрытия

удаляет лишь единицы узлов на каждом уровне. Экономия t_{plan} от сокращения вызовов процедуры построения путей составляет 3,4%, что соответствует удалению $\sim 3\%$ узлов. Более агрессивное отсечение ($\alpha < 100$) рискует удалить узлы, через которые проходит оптимальный план, поскольку оценки кардинальностей на промежуточных уровнях ненадёжны.

3. Процедура построения путей остаётся узким местом. Свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на перечисление или сортировку узлов. Замена алгоритма перечисления и оптимизация порядка обработки не могут существенно сократить число вызовов этой процедуры, потому что число допустимых разбиений определяется топологией графа соединений.

Для существенного улучшения необходимо сократить пространство поиска — уменьшая число отношений, подаваемых на вход DP, путём предварительного связывания подграфов с предсказуемыми свойствами. Этот вывод мотивирует переход к подходу со связыванием якорных участков.

Результаты на JOB-Complex. На JOB-Complex: $t_{\text{plan}} = 1,04$, $t_{\text{exec}} = 0,82$, $t_{\text{total}} = 0,83$, #лучше 10 из 30. Улучшение t_{exec} на 18% объясняется тем, что поуровневая материализация и сортировка кандидатов частично восстанавливают инварианты DPsize, но не устраняют значительные ошибки оценок на строковых соединениях. На запросах с умеренными ошибками (Q8, Q9, Q13, Q14) DPhypMod показывает результаты, сопоставимые с DPsize. На запросах с выраженной деградацией (Q5, Q11, Q12, Q20) он не достигает уровня GOO_{card} и GOO_{comb} , поскольку отсечение по $|\widehat{S}|$ на промежуточных уровнях неэффективно, когда сами оценки ошибочны на порядки.

4.6. Подход 5. Связывание якорных участков

Мотивация. Исходя из аннотации и выводов с прошлых подходов следует, что адаптивное планирование имеет смысл рассматривать как *предобработка графу* перед основным DP: сокращение числа вершин за счёт стягивания подграфов с надёжными оценками кардинальностей, после которой стандартный DPsize работает быстрее и устойчивее. Переключение между стратегиями происходит по единственному критерию $\#csg$, естественным образом отражающему обещанную в аннотации «сложность графа»: для простых графов применяется прямой DPsize, для сложных — связывание с последующим DPsize.

При этом связывание должно быть *безопасной*: стягиваться могут только те подграфы, для которых оценки кардинальностей заведомо надёжны, а центральные вершины не могут быть изолированы от своих соседей (это требование — прямое следствие провала подхода 2).

Определения. Пусть задан граф соединений $G = (\mathcal{R}, \mathcal{E})$ с n вершинами.

- **Якорная вершина** — вершина $v \in \mathcal{R}$, удовлетворяющая двум условиям:

1. $|v| < R_{\max}$

2. $|v| \cdot \alpha < \max_{u \in N(v)} |u|$

Якорь — это малая таблица, имеющая хотя бы одного соседа, который в α раз больше. Соединение якоря с крупным соседом высокоселективно, а его кардинальность оценивается надёжно, поскольку на единичных отношениях оценки более точные.

- **Якорный участок** — связная компонента подграфа, включающая в

себя якорную вершину и её соседей. Участок имеет ограниченную кардинальность:

1. $|v \cup N(v)| \leq L$, где v - якорная вершина

Если якорные участки смежны, т.е существует условие соединения между двумя вершинами из разных участков, то два участка объединяются в один.

Алгоритм. Входом является список базовых отношений и граф соединений. Алгоритм реализует каскад из трёх стратегий с убывающей точностью:

Шаг 1. Построение графа и оценка сложности. Строится граф соединений G . Для каждой связной компоненты C в G вычисляется $\#csg(C)$ — число связных подграфов, определяющее объём DP-таблицы и сложность перебора. Если $\#csg(C) \leq \tau$, граф передаётся стандартному DPsize без модификаций: при малом $\#csg$ точный перебор быстр.

Шаг 2. Связывание якорных участков. Если $\#csg(G) > \tau$, выполняется поиск и связание N итераций. Если на каком-то шаге условие перестает выполняться, то происходит досрочный выход из цикла итераций:

1. Для каждой вершины проверяется якорное условие. Все якоря и их соседи маркируются.
2. Удаление небезопасных вершин. Если вершина не является якорем, но входит в участок и содержит строковое соединение, то она удаляется из участка
3. Маркированные вершины разбиваются на связные компоненты обходом в ширину по подграфу маркированных вершин. Каждая связная компонента образует один якорный участок.

4. Каждый участок планируется *стандартным DPsize*.
5. Если кардинальность результата планирования участка превышает порог L , связывание участка отменяется: вершины возвращаются в граф как отдельные отношения.
6. Перестроение графа. Успешно спланированные участки заменяются новыми вершинами. Строится новый граф соединений и пересчитывается $\#csg$.

После связывания $\#csg$ пересчитывается. Если $\#csg \leq \tau$, новый граф передаётся DPsize.

Шаг 3. Жадный алгоритм. Если после связывания $\#csg > \tau$, оставшийся граф планируется комбинированным жадным алгоритмом GOO_{comb} .

Если граф имеет несколько компонент связности, они объединяются тем же GOO_{comb} в режиме декартова произведения.

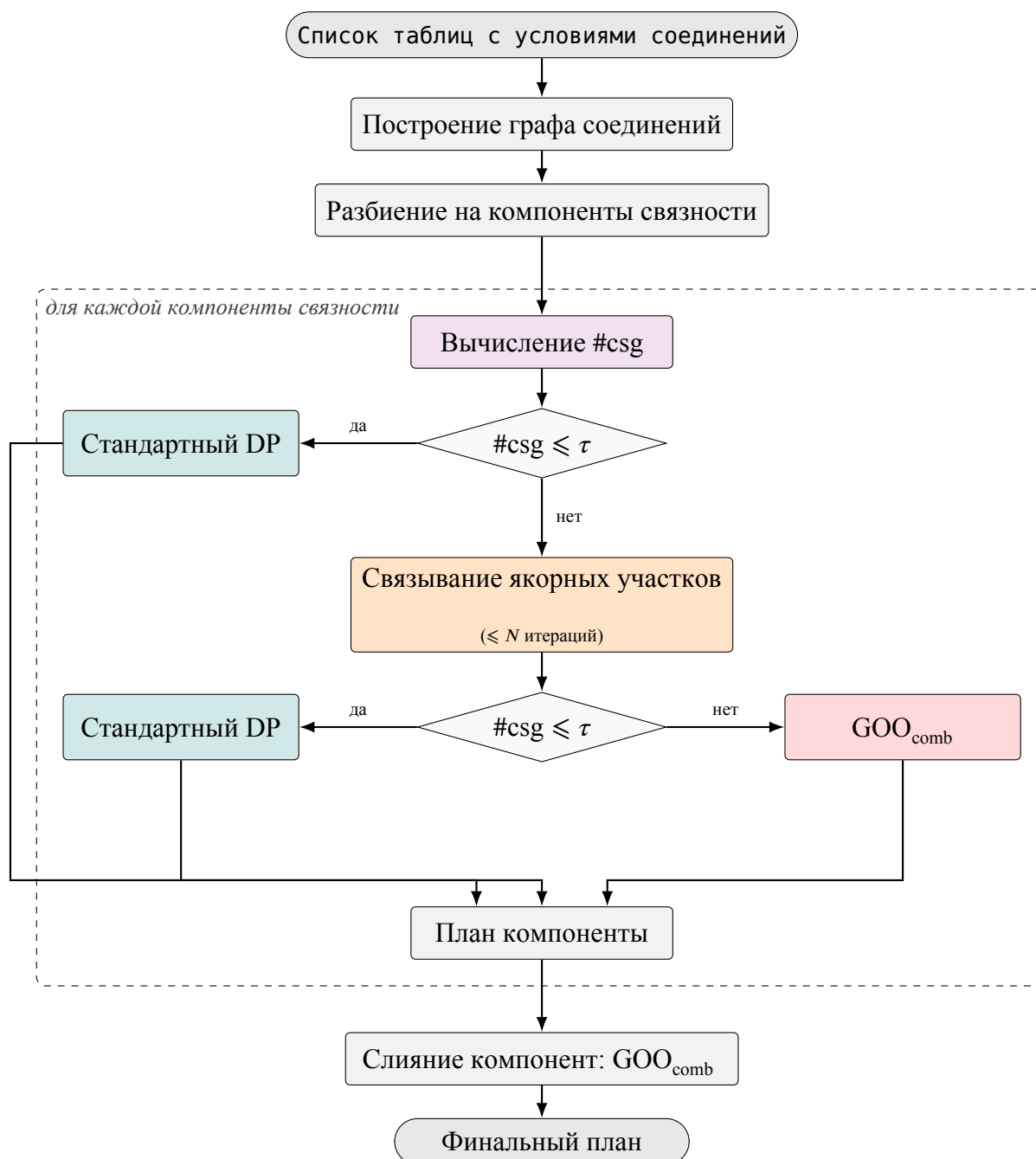


Рис. 4.1: Схема эвристического поиска порядка соединений: маршрутизация компонент связности по сложности графа #csg.

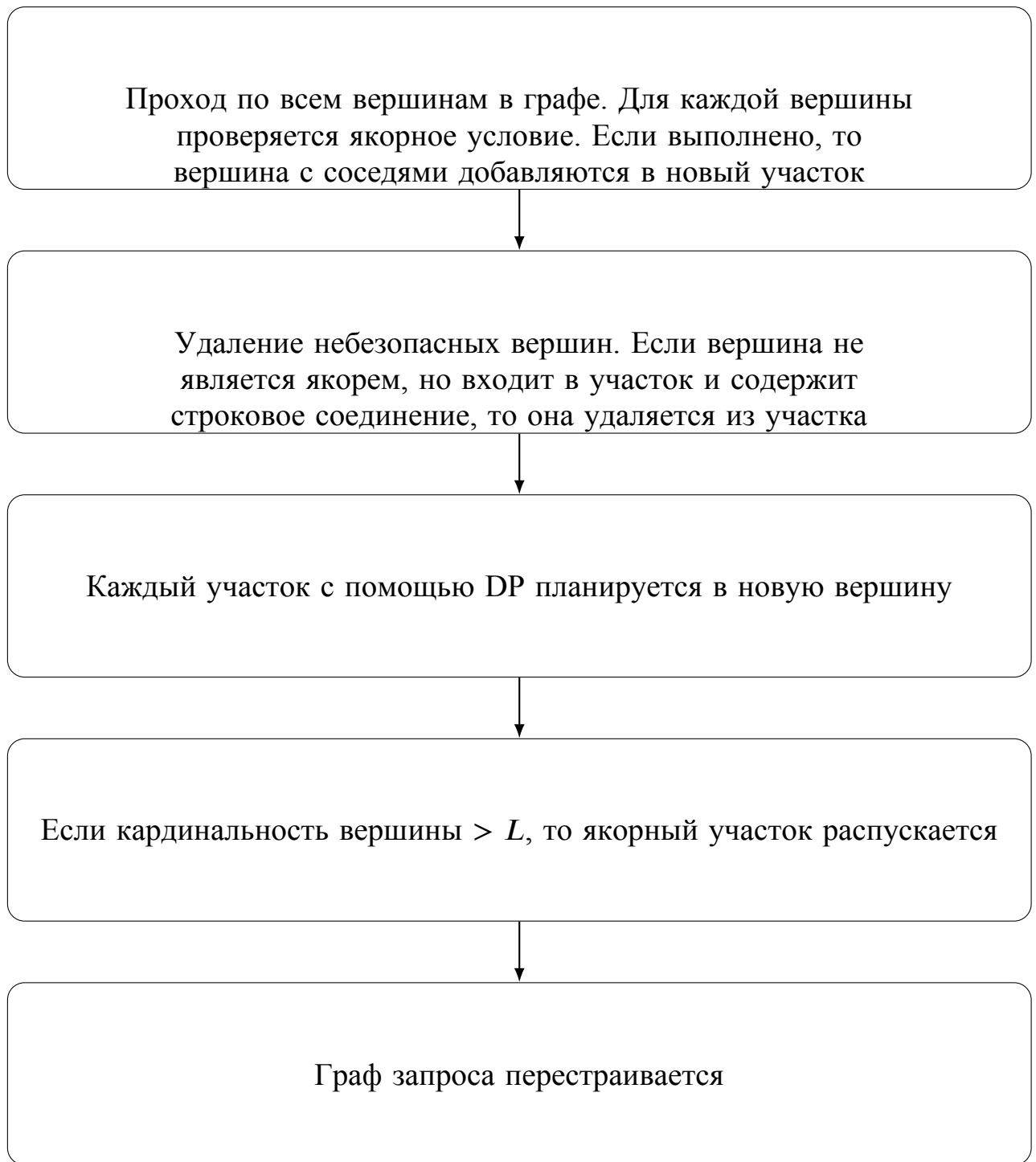


Рис. 4.2: Поиск и связывание якорных участков

Выбор параметров. Подход использует четыре параметра: $\tau = 350$, $R_{\max} = 50\,000$, $\alpha = 100$, $L = 100\,000$.

Порог $\tau = 350$ определяет границу переключения между точным DP и якорным связыванием.

Значение $R_{\max} = 50\,000$ отсекает все таблицы, сопоставимые по размеру с таблицами фактов для аналитических схем (10^5 – 10^7 строк). Множитель $\alpha = 100$ требует двухпорядкового различия между якорем и его крупнейшим соседом, ограничивая якоря справочниками и малыми таблицами, для которых соединения высокоселективны. Порог связывания $L = 100\,000$ обеспечивает, что новая вершина не вносит в граф отношение с большой кардинальностью. Стандартный DP внутри участка выбирает оптимальный план с высокой вероятностью, поскольку стоимостная модель надёжна для малых подпланов (см. секцию «Оценки в модели стоимости»). Проверка $|v \cup N(v)| \leq L$ является дополнительной защитой: если результат связывания оказался неожиданно большим (признак ошибки оценки), связывание отменяется.

Все четыре значения подобраны экспериментально на JOB. Дополнительная проверка на вариациях $\tau \in \{250, 300, 350\}$, $L \in \{25\,000, 50\,000, 100\,000\}$, $\alpha \in \{50, 100, 200\}$ и $R_{\max} \in \{25\,000, 50\,000, 100\,000\}$ показала, что суммарное отношение t_{total} к стандартному DPsize остаётся в узком диапазоне 0,86–0,89 на JOB и 0,11–0,12 на JOB-Complex, то есть подход слабо чувствителен к конкретному выбору в окрестности выбранных значений.

Защита центральных вершин. Свойство алгоритма: якорное условие $|v| < R_{\max} \wedge |v| \cdot \alpha < \max_{u \in N(v)} |u|$ гарантирует, что центральные вершины (например, в IMDB таблица title с $|t| = 2,5 \cdot 10^6$) *никогда* не становятся

якорями. Центральная вершина может попасть в якорный участок только как сосед якоря — но в этом случае она входит в участок *вместе со всеми* соседями якоря, а не изолируется от части своих соседей. Это исключает ситуацию, приводящую к деградации у топологической декомпозиции.

Более того, в типичных запросах JOB центральная вершина имеет степень, превышающую число вершин в якорном участке. Поскольку участок включает только соседей якоря, центральная вершина остаётся связанной с незатронутыми вершинами и сохраняет все параметризованные пути при последующем DPsize.

Результаты. Суммарные отношения к стандартному DPsize на JOB (113 запросов):

- t_{plan} : 0,822 (улучшение 17,8%),
- t_{exec} : 0,935 (улучшение 6,5%),
- t_{total} : 0,893 (улучшение 10,7%).

Из 113 запросов 91 показал улучшение t_{total} в строгом смысле; с учётом ожидаемого разброса runtime в $\pm 2\%$ количество запросов, не уступающих DPsize, составляет 99. Время планирования улучшилось в 97 запросах, исполнения — в 84.

Абсолютные суммарные времена:

- DPsize: $t_{\text{plan}} = 52,3$ с, $t_{\text{exec}} = 88,2$ с, $t_{\text{total}} = 140,5$ с.
- Связывание якорных участков: $t_{\text{plan}} = 42,9$ с, $t_{\text{exec}} = 82,5$ с, $t_{\text{total}} = 125,4$ с.

Суммарное улучшение — 15,1 секунды на 113 запросах, из которых 9,3 с приходится на t_{plan} и 5,7 с — на t_{exec} .

Анализ улучшения t_{plan} . Связывание уменьшает число вершин

графа соединений, подаваемого на вход DPsize. Поскольку $\#csg(G)$ растёт экспоненциально с числом вершин, удаление даже 2–3 вершин может сократить $\#csg$ в несколько раз. Наибольшая экономия наблюдается на запросах с 13–17 таблицами:

- Группа запросов 29a/29b/29c (17 таблиц): t_{plan} снизилось в среднем с 2,64 с до 1,20 с (в 2,2 раза). Связывание сократило число вершин на 4–5, уменьшив $\#csg$ ниже порога $\tau = 350$.
- Группа 17a–17d (14 таблиц): t_{plan} снизилось с ~ 250 мс до ~ 260 мс на запрос, причём дополнительно выиграло t_{exec} за счёт более удачного порядка.

Для запросов с ≤ 10 таблицами $\#csg$ уже невелик, и экономия t_{plan} составляет единицы процентов.

Анализ улучшения t_{exec} . Улучшение t_{exec} на 6,5% не является целью подхода, но является его следствием. Связывание фиксирует высокоселективные соединения (якорь + соседи) до основного DP. Внутри участка стандартный DP находит оптимальный план, поскольку участки малы и оценки кардинальностей для первых соединений надёжны. Новая вершина, созданная связыванием, несёт точную оценку кардинальности и фиксирует выбор физического пути (например, параметризованное индексное сканирование внутри участка). Это решение *не подвергается пересмотру* на последующих уровнях DPsize, где ошибки каскадируются и стоимостная модель менее надёжна. Наиболее яркий пример — запрос Q29c: t_{exec} снизилось с 3,35 с до 84 мс (в 40 раз), поскольку якорный план избежал четырёхуровневого вложенного сканирования с промежуточным результатом в 268 тысяч строк, который DPsize выбрал из-за заниженной оценки кардинальности на средних уровнях при построении плана.

Анализ деградаций. Из 22 запросов JOB, на которых t_{total} ухудшилось, 20 имеют деградацию не более 12%, и суммарные потери составляют 2,9 секунды (против 18,0 секунды суммарного выигрыша на остальных 91 запросе). Наибольшие деградации сосредоточены в группе запросов 31a/31b/31c (14 таблиц): t_{total} ухудшилось на 11%–43% (+387, +503, +993 мс соответственно). Причина: связывание выделило участок, в котором локальный DPsize по-другому распределил физические операторы, что привело к замене Nash хэш соединения на менее эффективное вложенное сканирование на верхнем уровне.

Среди 16 запросов с деградацией t_{plan} причиной является накладной расход на построение графа соединений, вычисление #csg и планирование участков, который не окупается на запросах с $\text{\#csg} \leq \tau$, где стандартный DPsize и так быстр (типично 5–25 мс).

Результаты на JOB-Complex. На 30 запросах JOB-Complex суммарные отношения к стандартному DPsize:

- t_{plan} : 0,867 (улучшение 13,3%),
- t_{exec} : 0,073 (улучшение 92,7%),
- t_{total} : 0,117 (улучшение 88,3%).

Абсолютные времена: DPsize $t_{\text{total}} = 648,8$ с (из них 612,7 с на исполнение), якорный подход $t_{\text{total}} = 75,9$ с (из них 44,5 с на исполнение). Суммарная экономия — 572,9 секунды на 30 запросах, причём 568,2 с приходится на t_{exec} . В строгое улучшение t_{total} показали 22 запроса из 30.

Доминирующий вклад в выигрыш даёт запрос Q12: DPsize строит план с верхним вложенным сканированием по строковому предикату `chn.name_pcode_nf = ak.name_pcode_nf`, что приводит к промежуточному

результату в $\sim 6,5 \cdot 10^8$ строк и времени исполнения 546,9 с. Якорный подход связывает участок вокруг `companu_name` и заменяет вложенное соединение на хэш соединение по тому же предикату на ранней стадии плана, что снижает t_{exec} до 668 мс (в ~ 820 раз). Аналогичный механизм избегания вложенного сканирования по строковому предикату срабатывает на запросах Q5, Q11, Q17, Q20: t_{exec} снижается на 1–2 порядка.

Наибольшая оставшаяся деградация — Q21 (t_{total} : 4,79 с $\rightarrow$ 6,57 с, +37%): связывание выделило участок, для которого локальный DPsize выбрал план, проигрывающий на верхнем уровне основного DP.

Результаты на истинных кардинальностях. В ходе исследования были также получены данные в условиях истинных кардинальностей. Для каждого запроса перебирались все связанные подграфы, каждому такому подграфу сопоставлялся подзапрос, для него вычислялась кардинальность. Таким образом каждому запросу из JOB и JOB-Complex, соответствует множество кардинальностей для всех его подзапросов. Было разработано расширение, которое подставляет истинные значения при оценке кардинальностей в планировщик PostgreSQL. Для эффективного доступа используется хэш таблица.

На наборе JOB при истинных кардинальностях суммарное время t_{plan} сократилось с 47,5 с до 43,1 с (–9,3%), t_{exec} практически не изменилось (25,4 с против 26,0 с, +2,2%), а t_{total} — с 72,9 с до 69,0 с (–5,3%). Якорный подход не уступает стандартному DPsize (с учётом разброса $\pm 5\%$) на 96 запросах из 113. Оставшийся выигрыш сосредоточен в планировании: на больших графах (12–17 таблиц) связывание уменьшает $\#csg$ и сокращает фазу DP. На наборе JOB-Complex отношения составили 0,892 по t_{plan} , 0,574 по t_{exec} и 0,795 по t_{total} (абсолютная экономия 9,8 с из общего $t_{\text{total}} = 47,8$ с).

При истинных кардинальностях главная компонента выигрыша смещается: относительная экономия t_{plan} становится меньше, чем при статистических оценках (9,3% против 17,8% на JOB), поскольку истинные значения уменьшают абсолютное время DPsize. t_{exec} при этом перестаёт быть источником выигрыша на JOB: если стоимостная модель получает точные оценки, она находит хорошие порядки, и связывание уже не приносит дополнительной выгоды на уровне исполнения.

Иллюстративный пример — запрос Q29b из JOB: на истинных кардинальностях DPsize строит план за 2,56 с при $t_{\text{exec}} = 6$ мс; якорный подход — за 1,21 с при $t_{\text{exec}} = 35$ мс. Граф запроса содержит около 17 вершин с несколькими малыми справочниками-якорями (role_type, info_type, comp_cast_type, keyword); их соседи образуют непересекающиеся участки. После связывания #csg нового графа падает ниже порога τ , и DPsize завершает перебор за время, сравнимое с накладными расходами на сам каскад. Выигрыш по t_{total} — 1,32 с (−51%).

Наибольшая оставшаяся деградация на JOB-Complex при истинных кардинальностях — Q28 (t_{total} : 598 → 1059 мс, +77%): связывание создало виртуальную вершину, для которой основной DPsize выбрал вложенное сканирование вместо хэш соединения, доступного в исходном графе.

5. Заключение

5.1. Полученные результаты

1. Реализованные в ядре PostgreSQL алгоритмы планирования соединений:

- три варианта жадного алгоритма с разными критериями выбора следующего соединения — по стоимости, по кардинальности и по их комбинации;
- итеративная топологическая декомпозиция графа соединений с применением специализированных вариантов динамического программирования для топологий;
- модификация алгоритма DPhyp — DPhypMod;
- новый метод — *связывание якорных участков*, являющийся основным результатом работы.

2. Реализовано расширение планировщика PostgreSQL, выполняющее подстановку истинных кардинальностей всех связных подграфов запроса на этапе оценки промежуточных результатов. Расширение использовано как средство анализа влияния качества оценок кардинальностей на принимаемые решения.

3. Экспериментальная оценка на бенчмарках JOB и JOB-Complex при стандартных оценках кардинальностей PostgreSQL показала, что относительно штатного DPsize якорный подход:

- На JOB сократил суммарное время планирования на 17,8% (52,3 с → 42,9 с), времени исполнения — на 6,5% (88,2 с → 82,5 с), их суммы —

- на 10,7% (140,5 с → 125,4 с); улучшение получено на 91 запросе из 113;
- На JOB-Complex сократил суммарное время планирования на 13,3%, исполнения — на 92,7% (главным образом за счёт избегания вложенного сканирования по строковому предикату, приводящему к деградации, на запросе Q12), общее время — на 88,3% (648,8 с → 75,9 с); улучшение получено на 22 запросах из 30;
 - Ни на одном запросе из обоих бенчмарков не приводит к сильной деградации: на JOB наибольшая потеря составляет 43% (+503 мс на Q31b); на JOB-Complex наибольшая потеря составляет 37% (+1,78 с на Q21).

5. Экспериментальная оценка при истинных кардинальностях (подстановка точных значений во все промежуточные оценки планировщика) показала, что якорный подход не уступает DPsize: на JOB суммарное время сокращено на 5,3% (72,9 с → 69,0 с), на JOB-Complex — на 20,5% (47,8 с → 38,0 с).

6. Сопоставительные эксперименты с альтернативными подходами (жадные алгоритмы GOO, DPhyp, DPhypMod, итеративная топологическая декомпозиция) подтвердили: замена алгоритма перебора подпланов без сокращения пространства поиска и переработки функции построения и сравнения путей даёт нейтральный либо отрицательный результат; качество стоимостной модели является ограничивающим фактором, не устраняемым выбором алгоритма перебора.

5.2. Направления дальнейшей работы

1. Динамический подбор параметров для якорного подхода Подбор параметров во время планирования позволил бы сделать данный подход более гибким к разным классам запросов, схемам баз данных.

2. Обнаружение ненадёжных оценок Разработка лёгковесной эвристики, маркирующей соединения с потенциально ненадёжными оценками (строковые столбцы, отсутствие первичного или внешнего ключа в соединении), позволила бы использовать кардинальную метрику, а для надёжных — стоимостную, приближая реализацию к алгоритму $\mathcal{A}^*$, адаптивному не только по сложности графа, но и по качеству доступных статистик.

3. Использование стоимостных критериев и селективности для определения якорей и их участков на единичных отношениях.

4. Выработка других критериев и апробация для применения якорного подхода:

- Доля потенциальных якорных вершин.

$$\rho_a(G) = \frac{|\{v \in V : A_condition(v) \text{ выполнено}\}|}{|V|},$$

где $A_condition(v)$ — якорное условие.

- Доля покрытия графа якорными участками.

$$\rho_z(G) = \frac{|\bigcup Z_i|}{|V|},$$

где Z_i — якорный участок.

- Доля небезопасных или селективных рёбер.

$$\rho_e(G) = \frac{|\{ e \in E : \text{sel}(e) \leq \text{threshold_selectivity} \vee E_unsafe(e) \text{ выполнено} \}|}{|E|},$$

где $E_unsafe(e)$ — условие небезопасности.

- Стоимостные критерии для якорного условия.

Приложение А. Результаты при стандартных оценках

Таблица 5.1: Сравнение алгоритмов планирования на наборе JOB (часть 1: DPsize, якорный подход и базовые варианты GOO). Колонки «DP standard» содержат абсолютные значения в мс; колонки прочих подходов — отношения к соответствующим значениям DPsize. Якорный подход применён в стандартной конфигурации: $\tau=350$, $R_{\max}=50\,000$, $\alpha=100$, $L=100\,000$.

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
1a	133.7	39.7	173.4	19113	0.919	1.065	0.952	1.000	0.979	1.067	0.999	1.034	0.972	1.010	0.980	1.034
1b	124.9	38.5	163.3	20000	0.947	0.950	0.948	1.000	0.972	1.001	0.979	1.031	0.974	0.962	0.972	1.032
1c	134.0	40.5	174.5	19083	0.968	0.936	0.961	1.000	0.953	0.982	0.960	1.034	0.933	1.605	1.089	1.865
1d	121.2	37.6	158.8	20001	1.025	1.028	1.026	1.000	1.064	1.016	1.053	1.031	0.991	0.991	0.991	1.031
2a	72.3	223.9	296.2	2889	1.054	0.891	0.932	1.000	1.032	1.194	1.155	0.980	1.132	1.220	1.199	0.980
2b	79.5	216.4	295.8	2886	0.943	0.922	0.928	1.000	0.986	1.221	1.158	0.982	0.996	1.259	1.188	0.982
2c	85.6	205.3	290.9	2885	0.742	0.896	0.851	1.000	0.948	1.288	1.188	0.982	0.932	0.060	0.317	2.907
2d	85.5	274.8	360.3	2902	0.830	0.958	0.928	1.000	0.860	0.987	0.957	0.976	0.923	1.013	0.991	0.976
3a	72.4	78.3	150.8	14793	0.882	0.887	0.885	1.000	1.082	0.976	1.027	1.000	0.979	1.975	1.496	1.034
3b	83.2	49.8	133.1	14294	0.856	0.930	0.881	1.000	0.889	1.004	0.932	1.000	0.973	2.934	1.708	15.108
3c	78.8	129.5	208.3	15330	0.948	0.904	0.919	1.000	0.892	0.980	0.946	1.000	1.102	1.315	1.235	1.102
4a	133.5	47.0	180.5	14108	0.779	0.927	0.816	1.000	0.944	0.955	0.947	1.027	0.983	1.047	1.000	0.998
4b	122.9	32.6	155.5	14065	0.950	0.961	0.952	1.000	1.098	1.255	1.131	1.010	1.124	1.064	1.112	1.000
4c	133.9	52.7	186.6	14159	0.819	0.902	0.842	1.000	0.930	0.989	0.947	1.048	1.006	0.987	1.000	0.997
5a	61.6	46.6	108.2	35093	0.900	0.933	0.914	1.000	1.080	1.015	1.052	2.455	1.172	2.768	1.859	2.324
5b	69.8	44.4	114.2	34557	0.867	0.955	0.903	1.000	1.001	1.000	1.001	1.016	0.945	1.012	0.971	1.000
5c	70.9	62.0	132.9	63342	0.784	1.000	0.882	1.000	0.961	7.010	3.782	1.955	1.042	4.334	2.577	3.966
6a	109.6	14.2	123.9	2887	0.816	0.904	0.826	1.000	0.908	1.064	0.926	0.984	0.945	1.071	0.960	0.984
6b	100.9	96.8	197.7	5022	0.906	0.920	0.913	1.000	1.041	1.007	1.024	1.001	1.036	1.013	1.024	1.001

продолжение на следующей странице

Продолжение таблицы 5.1

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
6c	99.4	5.5	104.9	2802	1.121	0.875	1.107	1.000	1.013	0.996	1.012	1.002	1.022	1.050	1.023	1.002
6d	104.2	2162.9	2267.1	7491	0.981	0.967	0.967	1.000	1.026	1.016	1.016	0.827	1.007	0.056	0.100	10.885
6e	112.1	14.1	126.2	3109	0.903	0.912	0.904	1.000	0.916	1.050	0.931	0.949	0.934	1.062	0.948	0.949
6f	110.8	2196.2	2307.0	7515	1.013	0.972	0.974	1.000	0.945	1.011	1.007	0.826	0.976	1.015	1.013	0.826
7a	407.5	415.5	823.0	13448	0.885	0.941	0.913	0.992	0.942	1.160	1.052	0.740	1.002	3.891	2.461	3.728
7b	400.2	134.4	534.7	9083	0.903	0.915	0.906	0.998	0.979	1.038	0.994	0.924	1.001	1.634	1.160	5.520
7c	401.3	1385.4	1786.7	56115	0.940	0.976	0.968	1.000	1.031	1.361	1.287	1.171	1.024	3.054	2.598	1.206
8a	233.3	367.1	600.4	38209	0.893	0.892	0.892	1.000	1.025	1.010	1.016	1.001	1.006	0.947	0.970	1.196
8b	238.2	57.4	295.6	38305	0.889	0.907	0.893	1.000	0.996	1.817	1.156	1.134	1.007	0.927	0.991	1.009
8c	238.2	1844.7	2082.9	419825	0.825	0.976	0.958	1.003	0.988	0.983	0.984	1.332	1.016	2.538	2.364	1.654
8d	236.4	857.9	1094.3	419825	0.908	0.973	0.959	1.003	1.035	2.019	1.806	1.332	1.037	5.253	4.342	1.654
9a	247.2	76.4	323.6	82112	0.899	0.926	0.906	0.967	1.027	4.934	1.949	1.895	1.059	1.126	1.075	1.668
9b	252.6	413.9	666.5	72334	0.905	0.885	0.893	1.000	0.995	1.056	1.033	1.010	0.984	0.200	0.497	1.773
9c	243.9	151.3	395.2	94246	0.851	0.916	0.875	0.998	1.018	1.897	1.355	1.431	1.037	0.887	0.980	2.706
9d	243.0	1910.2	2153.1	180958	0.883	1.030	1.011	1.000	1.027	0.573	0.624	1.930	1.048	0.684	0.725	2.517
10a	93.9	182.6	276.5	35990	1.059	0.904	0.959	1.000	1.084	1.134	1.117	1.414	1.015	1.320	1.216	1.496
10b	106.9	82.5	189.5	32874	0.857	0.915	0.882	1.000	0.886	1.066	0.965	1.452	0.899	2.786	1.721	1.635
10c	115.1	2929.9	3045.0	427156	0.899	0.972	0.969	1.000	0.841	0.311	0.331	1.304	0.823	0.199	0.222	1.096
11a	167.5	33.5	201.0	2801	1.018	1.031	1.020	1.000	0.954	16.029	3.468	1.094	0.968	0.975	0.969	1.000
11b	173.8	19.6	193.4	2801	0.942	0.936	0.942	1.000	0.916	1.779	1.003	1.091	0.895	13.067	2.125	3.889
11c	163.3	83.5	246.8	3322	1.062	1.034	1.052	1.000	0.911	4.367	2.080	1.689	0.986	2.948	1.649	3.267
11d	152.8	72.0	224.8	3325	1.038	1.049	1.042	1.000	1.022	4.821	2.238	1.687	0.943	6.598	2.753	1.426
12a	298.5	163.4	461.9	16470	1.030	1.021	1.027	1.000	0.958	0.820	0.909	3.322	0.948	0.586	0.820	1.152

продолжение на следующей странице

Продолжение таблицы 5.1

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
12b	304.8	39.1	343.9	19211	1.013	1.023	1.014	1.000	0.954	2.502	1.130	3.121	0.912	1.214	0.946	2.683
12c	311.4	338.0	649.4	17392	0.993	0.979	0.986	1.000	0.913	0.793	0.851	3.351	0.937	0.867	0.900	1.125
13a	308.5	572.2	880.7	19793	0.949	1.037	1.005	1.000	0.966	0.270	0.514	2.233	0.935	0.260	0.496	2.233
13b	316.6	257.4	573.9	19206	1.002	1.008	1.005	1.000	0.932	0.320	0.658	3.071	0.921	0.251	0.620	2.532
13c	311.9	246.1	558.0	19205	0.992	0.990	0.991	1.000	0.934	0.220	0.619	2.887	0.920	0.169	0.589	2.509
13d	314.2	888.5	1202.7	19934	0.936	1.038	1.011	1.000	0.902	0.735	0.779	3.202	0.920	2.433	2.037	1.174
14a	337.9	138.2	476.1	3664	0.980	0.971	0.977	1.000	0.959	1.009	0.973	1.000	0.954	4.274	1.917	1.012
14b	332.6	57.2	389.8	2981	1.008	0.932	0.996	1.000	0.974	1.002	0.978	1.000	0.982	5.011	1.573	1.008
14c	340.3	254.5	594.7	3672	0.953	0.938	0.946	1.000	0.909	1.149	1.012	1.040	0.946	4.252	2.360	1.063
15a	544.7	108.5	653.3	43390	0.960	0.905	0.950	1.000	0.932	1.536	1.033	1.208	0.913	1.183	0.958	4.290
15b	542.8	13.4	556.2	6110	0.966	1.247	0.972	1.000	0.952	1.098	0.955	1.003	0.917	1.028	0.920	1.000
15c	554.2	741.2	1295.4	171236	0.932	0.925	0.928	1.000	0.933	1.730	1.389	0.779	0.903	0.168	0.483	1.253
15d	543.9	728.8	1272.7	166651	1.005	0.976	0.988	1.000	0.965	1.329	1.174	0.801	0.922	0.203	0.510	1.202
16a	426.2	86.2	512.4	2824	1.006	0.970	1.000	1.000	0.965	1.110	0.989	0.997	0.914	0.974	0.924	0.997
16b	408.8	8447.7	8856.5	3695	0.865	0.919	0.916	0.999	0.983	1.040	1.037	0.819	0.973	1.021	1.019	0.819
16c	405.3	796.9	1202.1	3135	1.054	1.055	1.055	1.000	0.958	0.974	0.969	0.921	0.970	0.975	0.974	0.921
16d	406.4	643.1	1049.5	3045	0.941	0.977	0.963	1.000	0.962	0.969	0.966	0.937	0.970	0.986	0.980	0.937
17a	279.6	4754.1	5033.6	3355	0.940	0.977	0.975	0.999	0.975	0.998	0.996	0.875	0.946	0.988	0.986	0.875
17b	279.3	2897.9	3177.1	3364	0.972	0.998	0.996	1.000	0.970	3.638	3.403	0.925	0.989	3.637	3.404	0.925
17c	281.4	2813.6	3095.0	3348	0.940	0.967	0.964	1.000	0.965	3.679	3.432	0.929	0.977	3.691	3.445	0.929
17d	287.4	2979.1	3266.5	3350	0.975	0.973	0.973	1.000	0.929	3.550	3.319	0.928	0.956	3.621	3.386	0.928
17e	286.5	4803.1	5089.6	3355	0.917	0.952	0.950	0.999	0.948	0.995	0.993	0.875	0.952	1.003	1.000	0.875
17f	286.2	4142.4	4428.6	3556	0.945	1.007	1.003	1.001	0.933	2.618	2.509	0.875	0.961	2.655	2.545	0.875

продолжение на следующей странице

Продолжение таблицы 5.1

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
18a	356.4	1481.6	1838.0	23407	0.888	0.884	0.885	1.000	0.933	0.997	0.985	1.079	0.882	0.070	0.227	6.642
18b	376.0	159.6	535.5	16974	0.843	0.850	0.845	1.000	0.882	0.946	0.901	1.001	0.890	0.763	0.852	1.119
18c	374.9	1735.0	2109.9	22341	0.851	0.883	0.877	1.000	0.897	1.024	1.001	1.105	0.883	1.511	1.399	7.970
19a	432.8	83.6	516.4	82114	0.875	0.924	0.883	0.967	0.907	4.383	1.469	1.895	0.888	0.954	0.899	1.000
19b	405.0	42.9	447.9	48124	0.916	0.744	0.899	1.000	0.983	2.681	1.146	1.829	0.963	0.834	0.951	1.000
19c	423.5	164.6	588.1	94201	0.964	0.933	0.955	0.998	0.924	1.785	1.165	1.446	0.887	26.875	8.159	2.022
19d	418.3	4236.4	4654.6	180091	0.918	0.909	0.910	1.000	0.925	0.383	0.432	2.830	0.909	0.331	0.383	1.490
20a	368.3	1135.2	1503.5	5021	0.891	0.966	0.947	1.000	0.951	7.419	5.834	2.364	0.976	1.017	1.007	1.001
20b	371.3	860.0	1231.4	5020	0.987	0.956	0.965	1.000	0.923	3.518	2.736	1.894	0.942	0.652	0.740	1.001
20c	372.4	494.4	866.8	4694	0.923	0.913	0.917	1.000	0.883	1.800	1.406	1.002	0.920	0.993	0.961	1.001
21a	345.3	36.6	381.8	2802	0.973	0.996	0.975	1.000	0.928	8.969	1.698	1.094	0.945	1.186	0.968	1.000
21b	336.2	34.0	370.1	2802	0.933	0.956	0.935	1.000	0.954	3.276	1.167	1.094	0.971	1.056	0.978	1.000
21c	340.0	35.7	375.7	2802	0.985	0.983	0.985	1.000	0.934	22.323	2.965	1.098	0.964	0.970	0.964	1.000
22a	608.6	196.3	804.9	3671	0.952	1.217	1.016	1.003	0.921	2.851	1.392	1.174	0.935	20.725	5.761	1.262
22b	614.6	118.4	733.0	3669	0.840	0.884	0.847	1.002	0.899	3.968	1.395	1.174	0.921	34.890	6.406	1.262
22c	606.6	709.7	1316.3	3674	0.921	1.017	0.972	1.005	0.923	1.512	1.240	1.176	0.934	6.308	3.831	1.268
22d	590.0	1272.1	1862.1	3674	0.881	0.363	0.527	1.001	0.941	1.278	1.171	1.179	0.964	3.767	2.879	1.268
23a	598.2	206.0	804.2	45781	0.888	1.761	1.110	4.860	0.914	1.015	0.940	1.189	0.937	0.982	0.949	3.861
23b	593.8	18.9	612.7	3663	0.891	1.514	0.910	1.801	0.926	1.798	0.953	1.088	0.939	10.735	1.242	48.123
23c	609.9	285.9	895.8	48608	0.894	1.453	1.067	4.628	0.930	2.240	1.348	2.839	0.905	0.705	0.841	3.637
24a	725.7	266.9	992.6	3335	0.846	2.256	1.217	1.000	0.842	0.929	0.865	1.002	0.851	1.198	0.944	1.006
24b	711.2	54.4	765.5	3664	0.888	0.770	0.879	1.000	0.862	0.784	0.856	1.000	0.890	1.470	0.931	14.956
25a	644.5	1193.5	1838.0	4019	0.911	1.008	0.973	1.000	0.945	1.044	1.009	1.019	0.981	0.967	0.972	1.009

продолжение на следующей странице

Продолжение таблицы 5.1

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
25b	637.5	135.2	772.7	4005	0.937	0.961	0.941	1.000	0.971	1.035	0.982	1.000	1.008	0.616	0.940	12.540
25c	660.3	3422.1	4082.5	4707	0.872	0.909	0.903	1.000	0.919	0.951	0.945	1.022	0.962	0.967	0.966	1.013
26a	765.7	312.7	1078.4	4695	0.816	0.637	0.764	1.629	0.887	1.252	0.993	1.003	0.928	0.903	0.920	1.005
26b	760.9	159.1	920.0	3660	0.847	0.323	0.755	1.811	0.899	0.506	0.831	1.003	0.934	0.489	0.857	1.003
26c	768.0	532.6	1300.6	4695	0.884	0.809	0.853	1.629	0.890	1.681	1.214	1.003	0.907	0.953	0.926	1.001
27a	803.1	12.1	815.2	1022	0.785	1.372	0.794	3.382	0.879	0.675	0.876	1.419	0.883	4.445	0.936	2.838
27b	796.8	7.9	804.8	1020	0.841	5.560	0.887	7.342	0.855	0.711	0.854	1.359	0.887	6.803	0.945	2.842
27c	779.6	19.2	798.8	783	0.827	1.283	0.837	4.998	0.905	0.430	0.893	1.220	0.925	3.751	0.993	3.582
28a	1260.0	317.8	1577.8	3666	0.772	0.791	0.776	1.825	0.806	2.754	1.199	1.445	0.814	4.482	1.553	1.450
28b	1218.9	141.6	1360.5	3666	0.773	2.000	0.901	1.825	0.797	5.343	1.270	1.445	0.849	66.701	7.701	1.752
28c	1217.4	439.2	1656.6	3664	0.810	0.836	0.817	1.808	0.809	1.272	0.932	1.161	0.845	3.268	1.487	1.162
29a	2531.6	941.7	3473.3	2813	0.496	0.796	0.577	17.836	0.502	0.017	0.371	1.001	0.531	0.211	0.445	34.510
29b	2640.5	924.3	3564.8	2813	0.467	0.761	0.544	17.836	0.473	0.012	0.354	1.001	0.511	0.211	0.433	34.510
29c	2919.4	3405.1	6324.6	2813	0.414	0.025	0.204	1.000	0.439	0.014	0.210	1.000	0.443	0.032	0.222	1.000
30a	1139.0	1250.9	2389.9	4686	0.895	0.515	0.697	1.630	0.932	1.601	1.282	1.764	0.959	1.713	1.354	1.073
30b	1138.9	152.5	1291.4	4685	0.908	0.332	0.840	6.874	0.941	1.303	0.984	1.738	0.946	0.650	0.911	11.525
30c	1141.7	1230.0	2371.6	4683	0.857	0.881	0.869	1.640	0.910	4.746	2.900	1.764	0.936	1.409	1.181	1.019
31a	984.1	2522.6	3506.7	4708	0.847	1.213	1.110	1.035	0.969	0.417	0.572	1.044	0.955	0.048	0.303	2.123
31b	979.3	180.3	1159.7	4691	0.885	4.418	1.432	2.351	0.971	1.022	0.979	1.000	0.992	0.204	0.870	1.370
31c	976.9	3756.5	4733.4	4708	0.878	1.296	1.209	1.037	0.964	0.295	0.433	1.044	0.973	0.054	0.243	2.323
32a	42.3	4.9	47.2	2809	1.621	0.911	1.534	1.002	0.854	1.778	0.951	1.238	1.146	0.998	1.131	1.000
32b	47.7	59.4	107.2	2809	0.634	0.929	0.786	1.002	0.750	1.580	1.210	1.238	1.024	1.003	1.012	1.000
33a	612.0	19.2	631.2	3257	0.580	0.766	0.585	1.053	0.655	1.233	0.673	1.127	0.637	0.345	0.628	3.178

продолжение на следующей странице

Продолжение таблицы 5.1

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, mc	exec, mc	total, mc	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
33b	593.3	18.6	611.9	2450	0.700	0.627	0.698	1.028	0.644	1.061	0.657	1.037	0.671	0.408	0.663	2.975
33c	548.0	17.8	565.8	3295	0.672	0.743	0.675	1.037	0.707	2.054	0.750	1.668	0.736	2.838	0.802	21.248

Таблица 5.2: Сравнение алгоритмов планирования на наборе JOB (часть 2: комбинированный GOO, топологическая итеративная декомпозиция и варианты DPhyp). Все значения — отношения к соответствующим значениям DPsizе.

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp _{mod}				DPhyp			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
1a	0.905	1.030	0.934	1.000	0.974	11.353	3.351	5.607	0.897	0.971	0.914	1.000	0.926	1.009	0.945	1.000
1b	0.928	0.987	0.942	1.000	1.022	8.282	2.732	4.867	0.926	0.975	0.937	1.000	0.989	0.973	0.985	1.000
1c	0.840	0.972	0.871	1.001	0.933	3.966	1.637	4.919	0.856	0.911	0.869	1.000	1.003	0.936	0.987	1.000
1d	0.896	0.980	0.916	1.036	1.015	11.834	3.578	5.000	0.959	1.002	0.969	1.000	0.977	0.968	0.975	1.000
2a	1.116	1.144	1.137	0.980	1.133	6.728	5.362	84.680	0.998	1.126	1.095	0.980	1.216	1.167	1.179	0.980
2b	0.911	1.185	1.112	0.982	0.990	7.112	5.467	84.514	0.850	1.181	1.093	0.982	1.029	1.214	1.164	0.982
2c	0.894	0.387	0.536	2.966	0.962	6.968	5.201	83.544	0.886	1.247	1.141	0.982	0.970	1.276	1.186	0.982
2d	0.839	0.954	0.927	0.976	0.985	6.482	5.178	86.854	0.898	0.961	0.946	0.976	0.850	0.983	0.952	0.976
3a	1.124	0.939	1.028	1.000	1.089	6.700	4.005	21.775	1.178	0.917	1.042	1.000	1.199	0.931	1.060	1.000
3b	0.885	0.953	0.911	1.000	1.011	2.420	1.538	14.211	0.769	0.936	0.832	1.000	0.980	0.931	0.961	1.000
3c	1.022	0.973	0.992	1.000	1.034	4.833	3.395	20.626	0.965	0.954	0.958	1.000	0.963	0.964	0.964	1.000
4a	0.934	0.883	0.921	1.027	1.042	10.559	3.520	7.751	0.962	0.958	0.961	1.000	1.007	0.964	0.996	1.000
4b	1.056	1.165	1.079	1.010	1.128	5.850	2.118	7.462	1.090	0.973	1.065	1.000	1.069	0.972	1.049	1.000
4c	0.961	0.930	0.952	1.048	1.055	19.881	6.371	8.089	0.950	0.939	0.947	1.000	0.967	0.936	0.959	1.000
5a	0.956	0.961	0.958	1.000	1.139	12.541	6.049	8.569	0.932	1.008	0.965	1.000	1.156	0.976	1.078	1.000
5b	0.838	0.981	0.894	1.000	1.064	0.787	0.956	8.068	0.841	0.968	0.890	1.000	1.007	0.990	1.000	1.000
5c	0.946	1.010	0.976	0.999	1.024	9.313	4.890	4.577	0.966	1.006	0.984	0.999	1.019	1.015	1.017	0.999
6a	0.929	0.973	0.934	0.984	1.059	4.732	1.481	27.472	0.926	0.959	0.929	0.984	0.910	0.970	0.917	0.984
6b	1.015	0.969	0.992	1.001	1.083	1.254	1.167	16.235	1.022	0.954	0.989	1.001	1.016	0.969	0.993	1.001
6c	1.046	0.960	1.042	1.002	1.143	11.504	1.691	28.311	1.030	0.956	1.026	1.002	1.038	1.068	1.039	1.002
6d	0.913	0.055	0.095	10.885	1.061	0.054	0.101	10.885	1.008	0.985	0.986	0.826	1.074	0.991	0.995	0.826

продолжение на следующей странице

Продолжение таблицы 5.2

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp _{mod}				DPhyp			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
6e	0.908	0.999	0.918	0.949	0.979	4.579	1.382	25.510	0.872	0.964	0.882	0.946	0.951	0.969	0.953	0.946
6f	0.974	0.983	0.983	0.826	1.045	0.799	0.811	58.105	0.942	0.980	0.979	0.827	0.924	0.986	0.983	0.825
7a	0.986	0.708	0.846	4.597	1.072	0.102	0.583	4.588	0.912	0.937	0.924	0.742	1.015	0.963	0.989	0.739
7b	0.992	0.528	0.875	6.803	1.103	0.433	0.934	6.794	0.972	0.965	0.970	0.925	1.010	0.938	0.992	0.923
7c	1.006	0.924	0.943	1.627	1.094	0.588	0.702	1.896	0.960	0.973	0.970	0.998	1.010	1.037	1.031	0.998
8a	0.982	0.963	0.970	1.000	1.065	1.447	1.298	4.028	0.925	0.898	0.908	1.195	0.974	0.905	0.932	1.000
8b	0.913	0.938	0.918	1.000	1.036	2.470	1.314	3.264	0.975	0.885	0.957	1.009	1.009	0.959	1.000	1.000
8c	0.954	0.925	0.928	1.011	1.045	1.956	1.852	1.507	0.928	0.999	0.991	0.995	0.987	0.999	0.998	0.982
8d	0.993	1.024	1.017	1.011	1.134	4.228	3.559	1.507	0.934	1.003	0.989	0.995	0.997	1.021	1.016	0.982
9a	0.959	4.437	1.780	1.883	1.116	4.157	1.833	1.983	0.916	0.969	0.928	1.000	1.008	0.982	1.002	1.000
9b	0.960	1.134	1.068	2.098	1.069	0.563	0.755	2.236	0.920	0.173	0.456	1.120	1.000	0.908	0.943	0.999
9c	0.983	1.705	1.260	1.378	1.057	4.125	2.232	1.876	0.922	0.951	0.933	0.998	0.991	0.980	0.987	0.998
9d	0.920	0.760	0.778	1.764	1.105	1.088	1.090	1.856	0.956	0.969	0.968	0.970	0.927	0.919	0.920	0.970
10a	0.942	1.265	1.155	1.083	1.040	14.430	9.883	6.311	0.972	0.951	0.958	1.001	0.944	0.883	0.904	1.000
10b	0.904	1.029	0.958	1.000	0.900	26.447	12.027	7.903	0.886	1.001	0.936	1.000	0.927	0.898	0.914	0.999
10c	0.794	0.224	0.245	1.102	0.787	0.251	0.271	1.117	0.825	0.975	0.969	1.000	0.957	0.972	0.971	0.998
11a	0.846	0.980	0.868	1.090	0.885	0.863	0.882	2.513	0.997	0.951	0.990	1.000	0.969	0.953	0.966	1.000
11b	0.815	1.649	0.899	1.089	0.826	0.487	0.792	2.189	0.860	0.929	0.867	1.000	0.941	0.903	0.937	1.000
11c	0.846	1.426	1.042	1.201	0.853	3.382	1.708	5.923	0.937	0.943	0.939	1.000	1.038	0.961	1.012	1.000
11d	0.926	1.738	1.186	1.200	0.982	2.942	1.610	5.918	0.957	0.992	0.969	1.000	1.031	0.969	1.011	1.000
12a	0.893	1.350	1.055	1.004	0.902	1.070	0.962	1.025	0.904	0.903	0.904	0.999	0.983	0.902	0.954	0.999
12b	0.850	0.961	0.862	1.000	0.843	0.949	0.855	1.000	0.924	0.996	0.932	1.000	0.979	0.945	0.975	1.000
12c	0.906	1.288	1.105	1.004	0.852	0.666	0.755	1.019	0.888	0.891	0.889	0.997	0.974	0.895	0.933	0.997

продолжение на следующей странице

Продолжение таблицы 5.2

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp _{mod}				DPhyp			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
13a	0.921	1.185	1.092	1.006	0.905	14.738	9.893	8.186	0.911	0.931	0.924	0.994	0.975	0.945	0.956	0.994
13b	0.886	0.937	0.909	1.000	0.888	0.249	0.601	2.536	0.877	0.923	0.898	1.000	0.992	0.960	0.978	1.000
13c	0.935	0.957	0.945	1.000	0.868	0.171	0.561	2.509	0.930	0.922	0.926	1.000	0.969	0.943	0.957	1.000
13d	0.916	0.976	0.960	0.990	0.882	9.434	7.200	8.128	0.916	0.947	0.939	0.990	0.922	0.947	0.940	0.990
14a	0.915	1.035	0.950	1.000	0.917	1.317	1.033	17.705	0.887	0.943	0.903	1.000	1.004	0.955	0.990	1.000
14b	0.953	0.940	0.951	1.000	0.916	0.670	0.880	16.730	0.928	0.912	0.926	1.000	0.973	0.920	0.965	1.000
14c	0.902	0.983	0.936	1.001	0.912	3.384	1.970	42.870	0.873	0.915	0.891	1.000	0.950	0.924	0.939	1.000
15a	0.907	1.134	0.945	2.151	0.897	1.072	0.926	1.039	0.907	0.920	0.909	0.991	0.941	0.898	0.934	0.991
15b	0.926	1.166	0.932	1.000	0.869	8.050	1.042	7.356	0.889	1.094	0.893	1.000	0.975	1.009	0.975	1.000
15c	0.916	1.218	1.089	1.676	0.876	3.220	2.218	0.990	0.925	1.200	1.083	0.730	0.961	1.666	1.364	0.792
15d	0.945	1.266	1.129	1.670	0.905	3.366	2.314	0.999	0.960	1.263	1.133	0.742	0.998	1.244	1.139	0.749
16a	0.944	0.951	0.945	0.997	0.918	4.646	1.545	83.408	0.940	0.942	0.940	0.997	0.955	0.924	0.950	0.997
16b	0.960	0.994	0.992	0.819	1.020	6.065	5.832	180.107	0.940	0.959	0.958	0.817	0.966	0.961	0.961	0.816
16c	0.950	0.950	0.950	0.921	1.079	4.727	3.498	187.410	0.974	0.965	0.968	0.920	0.969	0.976	0.974	0.920
16d	0.919	0.937	0.930	0.937	1.071	4.413	3.119	188.914	0.931	0.966	0.952	0.941	0.974	0.982	0.979	0.941
17a	0.912	0.954	0.952	0.875	0.941	2.279	2.204	0.928	0.918	0.953	0.951	0.875	0.968	0.979	0.979	0.875
17b	0.947	3.451	3.231	0.925	0.902	3.466	3.241	0.920	0.931	0.998	0.993	0.921	0.937	1.002	0.996	0.913
17c	0.945	3.525	3.290	0.929	0.909	3.526	3.288	0.925	0.939	0.991	0.986	0.918	0.958	1.003	0.999	0.917
17d	0.905	3.403	3.183	0.928	0.865	3.376	3.155	0.924	0.889	0.964	0.957	0.918	0.898	0.963	0.957	0.916
17e	0.924	0.947	0.945	0.875	0.874	3.001	2.881	0.989	0.917	0.949	0.947	0.877	0.918	0.951	0.949	0.875
17f	0.925	2.514	2.411	0.875	0.881	2.687	2.570	0.878	0.895	2.486	2.383	0.875	0.937	2.611	2.503	0.875
18a	0.883	0.248	0.371	7.398	0.877	5.936	4.955	1.991	0.917	0.933	0.930	0.988	0.952	0.924	0.930	0.988
18b	0.847	0.879	0.856	1.000	0.839	0.868	0.848	1.000	0.830	0.872	0.842	1.000	0.883	0.858	0.876	1.000

продолжение на следующей странице

Продолжение таблицы 5.2

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp _{mod}				DPhyp			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
18c	0.866	0.991	0.969	1.076	0.879	0.902	0.898	1.046	0.873	0.905	0.899	1.000	0.942	0.913	0.918	1.000
19a	0.883	3.985	1.385	1.883	0.881	10.769	2.482	1.620	0.866	0.918	0.874	1.000	0.953	0.916	0.947	1.000
19b	0.936	2.338	1.070	1.813	0.906	0.842	0.900	1.000	0.938	0.804	0.925	1.000	1.007	0.833	0.990	1.000
19c	0.886	1.615	1.090	1.379	0.827	0.771	0.811	2.734	0.889	0.918	0.898	0.998	0.965	0.921	0.953	0.998
19d	0.915	0.499	0.537	1.873	0.889	0.607	0.633	2.593	0.892	0.926	0.923	0.974	0.950	0.917	0.920	0.973
20a	0.902	0.904	0.904	1.594	0.979	6.966	5.500	13.886	0.931	0.967	0.958	1.001	0.984	0.974	0.976	1.000
20b	0.913	0.555	0.663	1.594	0.984	24.909	17.694	12.243	0.924	0.960	0.949	1.001	0.949	0.621	0.720	1.000
20c	0.856	0.880	0.870	1.629	1.027	7.032	4.452	13.259	0.924	1.729	1.383	1.001	0.983	0.944	0.960	1.000
21a	0.882	17.905	2.513	1.093	1.005	1.877	1.089	2.471	0.902	1.098	0.920	1.000	1.008	1.164	1.023	1.000
21b	0.893	6.214	1.381	1.093	1.038	0.566	0.995	2.475	0.913	1.003	0.921	1.000	0.985	1.031	0.989	1.000
21c	0.860	1.074	0.880	1.091	1.039	2.696	1.197	3.658	0.951	1.151	0.970	1.000	1.000	1.226	1.022	1.000
22a	0.860	1.133	0.927	1.003	0.999	4.688	1.899	21.465	0.911	1.105	0.959	1.000	0.983	1.146	1.022	1.000
22b	0.860	0.938	0.872	1.002	0.961	2.014	1.131	20.049	0.881	0.900	0.884	1.000	0.944	0.919	0.940	1.000
22c	0.872	0.984	0.932	1.005	0.982	5.244	3.280	23.277	0.923	0.969	0.948	1.000	1.000	1.019	1.010	1.000
22d	0.872	0.373	0.531	1.001	0.991	4.377	3.304	42.490	0.957	1.082	1.042	1.000	0.955	1.084	1.043	1.000
23a	0.896	3.708	1.617	1.298	1.004	0.588	0.897	1.170	0.942	0.951	0.944	0.991	1.007	0.956	0.994	0.991
23b	0.897	1.564	0.918	1.801	0.970	6.084	1.128	14.602	0.938	1.319	0.950	1.000	0.987	1.349	0.998	1.000
23c	0.886	2.711	1.468	1.222	0.939	0.647	0.846	1.350	0.912	0.913	0.913	0.959	0.999	0.929	0.977	0.959
24a	0.839	2.202	1.205	1.000	0.883	6.997	2.527	66.533	0.870	0.890	0.875	1.000	0.952	0.895	0.937	1.000
24b	0.820	0.721	0.813	1.000	0.895	2.395	1.002	12.706	0.896	0.725	0.884	1.000	0.969	0.745	0.953	1.000
25a	0.932	1.012	0.984	1.029	0.964	1.211	1.125	1.011	0.910	1.021	0.982	1.000	0.985	1.025	1.011	1.000
25b	0.915	0.970	0.925	1.000	0.938	0.964	0.943	1.000	0.937	0.961	0.941	1.000	0.975	0.979	0.976	1.000
25c	0.879	0.959	0.946	1.037	0.918	1.530	1.431	1.029	0.902	0.937	0.931	1.000	0.911	0.941	0.937	1.000

продолжение на следующей странице

Продолжение таблицы 5.2

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp _{mod}				DPhyp			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
26a	0.854	0.683	0.804	1.629	0.920	13.661	4.615	11.178	0.900	0.932	0.909	1.000	0.961	1.493	1.115	1.000
26b	0.813	0.341	0.731	1.811	0.926	5.682	1.749	8.636	0.905	0.896	0.904	1.000	0.973	0.575	0.904	1.000
26c	0.859	0.843	0.853	1.629	0.935	32.239	13.753	12.074	0.898	2.970	1.746	1.001	0.958	0.987	0.970	1.000
27a	0.816	1.633	0.828	6.792	0.932	2.099	0.950	6.373	0.915	0.883	0.914	1.062	0.989	1.786	1.001	1.059
27b	0.834	5.951	0.884	7.342	0.876	1.292	0.880	4.783	0.918	0.761	0.917	1.025	0.998	2.735	1.015	1.061
27c	0.855	0.989	0.858	4.407	0.917	3.495	0.979	11.761	0.962	0.732	0.957	1.095	1.003	0.776	0.997	1.091
28a	0.769	0.795	0.774	1.825	0.767	15.014	3.637	12.651	0.940	0.899	0.932	1.000	1.014	0.957	1.002	1.000
28b	0.773	2.047	0.906	1.825	0.838	3.908	1.158	12.250	0.963	0.930	0.959	1.000	1.039	0.975	1.033	1.000
28c	0.793	0.804	0.796	1.808	0.828	17.006	5.118	12.626	0.977	0.919	0.961	1.000	1.023	0.938	1.000	1.000
29a	0.491	0.082	0.380	34.570	0.472	11.189	3.378	47.821	1.215	0.033	0.894	1.000	1.173	0.056	0.870	1.000
29b	0.463	0.076	0.363	34.570	0.468	5.486	1.769	47.821	1.160	0.029	0.867	1.000	1.126	0.052	0.848	1.000
29c	0.424	0.026	0.210	1.000	-0.000	-0.000	-0.000	-0.000	1.108	0.110	0.571	1.000	1.168	0.118	0.602	1.000
30a	0.878	0.528	0.695	1.630	0.978	0.730	0.848	1.007	0.958	0.761	0.855	1.000	0.994	0.294	0.628	1.000
30b	0.896	0.515	0.851	6.878	0.983	1.220	1.011	1.001	0.939	0.945	0.939	1.000	1.011	0.972	1.006	1.000
30c	0.892	0.901	0.897	1.640	0.935	3.096	2.056	1.009	0.954	0.966	0.960	1.000	1.010	1.106	1.060	1.000
31a	0.902	1.233	1.140	1.035	0.976	1.536	1.379	1.017	0.933	0.594	0.689	1.000	1.005	3.131	2.535	1.000
31b	0.909	4.509	1.468	2.351	0.945	1.014	0.956	1.000	0.947	0.945	0.947	1.000	0.980	0.955	0.976	1.000
31c	0.934	1.341	1.257	1.037	0.951	1.760	1.593	1.030	0.937	0.531	0.615	1.000	1.016	2.811	2.440	1.000
32a	0.844	1.764	0.940	1.238	0.865	60.177	7.052	25.424	0.856	1.011	0.872	0.998	0.903	0.996	0.913	0.998
32b	1.048	1.462	1.278	1.238	0.909	5.014	3.185	25.424	0.941	0.920	0.929	0.998	1.019	0.944	0.977	0.998
33a	0.613	0.672	0.615	1.053	0.657	21.478	1.291	16.673	1.080	0.776	1.071	0.990	1.147	1.200	1.149	0.990
33b	0.622	0.621	0.622	1.028	0.678	18.523	1.220	21.806	1.086	0.710	1.075	0.997	1.164	1.206	1.165	0.996
33c	0.698	0.754	0.700	1.037	0.723	49.747	2.264	16.747	0.977	0.910	0.975	0.982	1.084	1.896	1.109	0.981

Таблица 5.3: Сравнение алгоритмов планирования на наборе JOB-Complex (часть 1: DPsize, якорный подход и базовые варианты GOO).

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
1	356.3	37.6	393.9	2802	0.933	0.941	0.934	1.000	0.940	5.240	1.350	1.090	1.020	1.050	1.020	1.000
2	163.0	31.5	194.5	2801	0.945	1.056	0.963	1.000	1.070	15.500	3.400	1.090	1.050	1.060	1.060	1.000
3	322.9	34.6	357.5	2802	0.933	0.976	0.937	1.000	1.040	9.250	1.830	1.090	1.020	1.130	1.030	1.000
4	320.7	34.6	355.4	2802	1.046	1.043	1.046	1.000	1.040	9.150	1.830	1.090	1.020	1.110	1.020	1.000
5	2580.0	6071.9	8652.0	12924	0.597	0.070	0.224	1.235	0.590	0.170	0.300	1.250	0.600	0.110	0.260	1.000
6	313.7	32.6	346.3	2802	0.977	1.048	0.983	1.000	1.010	3.200	1.210	1.090	1.070	1.060	1.070	1.000
7	320.3	32.7	353.0	2802	1.046	1.008	1.043	1.000	1.040	3.230	1.250	1.090	1.030	1.060	1.040	1.000
8	1420.0	305.9	1725.9	55868	0.997	0.509	0.913	1.493	1.030	1.140	1.050	1.050	1.050	1.120	1.060	1.000
9	1139.1	277.6	1416.7	3666	0.890	0.871	0.886	1.825	0.920	11.470	2.980	1.850	0.950	0.960	0.950	1.000
10	476.1	1823.5	2299.6	28132	1.066	1.014	1.024	1.000	1.040	2.700	2.360	1.190	1.180	0.080	0.300	3.030
11	804.3	14425.6	15229.9	29350	0.882	0.049	0.093	1.101	0.840	18.260	17.340	1.130	0.970	0.060	0.110	1.000
12	2002.5	539307.3	541309.8	25817	0.768	0.001	0.004	1.156	0.710	0.510	0.510	1.160	0.800	0.000	0.000	1.040
13	1248.0	534.3	1782.3	20062	0.809	0.629	0.755	1.003	0.770	0.770	0.770	1.000	0.820	1.710	1.090	3.620
14	709.1	81.6	790.7	25319	0.886	0.945	0.892	1.000	0.900	0.940	0.900	0.990	0.960	8.930	1.780	3.810
15	1250.4	4080.8	5331.2	16310	0.996	0.994	0.995	1.000	0.680	4.900	3.910	3.420	1.010	0.060	0.280	10.790
16	340.6	1362.0	1702.6	23407	0.994	1.002	1.000	1.000	1.010	2.520	2.220	1.230	1.000	0.080	0.260	6.640
17	730.4	2163.8	2894.2	137242	0.811	0.233	0.379	1.071	0.820	0.250	0.390	1.000	0.820	0.330	0.450	1.020
18	545.2	3957.8	4503.0	16362	0.931	0.956	0.953	1.000	0.940	0.660	0.690	1.020	0.990	0.060	0.170	10.800
19	2929.5	3710.2	6639.7	68364	0.778	1.219	1.024	1.018	0.740	0.600	0.660	1.020	0.760	0.990	0.890	1.020
20	2365.6	2027.5	4393.1	3665	0.684	0.082	0.405	4.821	0.640	1.660	1.110	1.860	0.690	0.050	0.390	1.010
21	1615.9	3196.2	4812.1	55893	1.046	1.534	1.370	1.003	0.960	0.680	0.780	1.000	1.040	0.080	0.400	1.090

продолжение на следующей странице

Продолжение таблицы 5.3

Запрос	DP standard				Якорный				GOO _{cost}				GOO _{card}			
	plan, mc	exec, mc	total, mc	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
22	469.7	3640.1	4109.8	58678	1.027	1.050	1.047	1.000	0.990	0.990	0.990	0.980	0.960	0.110	0.210	2.320
23	2460.2	1192.4	3652.6	21100	0.663	0.541	0.623	1.191	0.620	221.180	72.620	1.200	0.650	0.590	0.630	1.050
24	4199.7	5343.5	9543.2	69367	1.051	1.033	1.041	0.999	0.650	3.770	2.400	1.020	0.670	3.060	2.010	1.020
25	1692.3	916.3	2608.7	27074	0.962	0.607	0.839	1.109	1.010	300.030	106.050	1.120	1.040	0.770	0.940	1.000
26	295.3	393.1	688.4	7114	0.998	0.993	0.995	1.000	1.030	1.130	1.080	1.000	1.030	1.080	1.060	1.000
27	1261.2	4442.3	5703.5	2889	0.872	0.979	0.955	1.000	1.070	1.110	1.100	0.980	1.070	1.090	1.090	0.980
28	572.4	812.7	1385.0	15158	1.010	0.887	0.938	12.792	1.060	2.740	2.050	1.220	1.040	0.270	0.590	11.660
29	2113.4	675.3	2788.7	22178	0.817	0.815	0.816	1.133	0.900	2.930	1.400	1.190	0.900	0.980	0.920	1.000
30	1192.0	4376.0	5568.0	2889	0.963	1.009	0.999	1.000	1.100	1.090	1.090	0.980	1.050	1.090	1.080	0.980

Таблица 5.4: Сравнение алгоритмов планирования на наборе JOB-Complex (часть 2).

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp				DPhyp _{mod}			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
1	1.140	10.570	2.040	1.090	0.930	1.580	1.000	2.420	1.020	1.130	1.030	1.000	0.980	1.120	1.000	1.000
2	1.110	1.030	1.090	1.090	1.040	0.920	1.020	2.510	1.060	1.020	1.050	1.000	0.960	1.050	0.980	1.000
3	1.190	18.760	2.890	1.090	1.000	1.960	1.090	2.470	0.990	1.190	1.010	1.000	0.990	1.170	1.010	1.000
4	1.160	18.480	2.840	1.090	0.980	1.880	1.070	2.470	0.990	1.170	1.010	1.000	0.980	1.180	1.000	1.000
5	0.600	0.070	0.230	1.230	0.590	0.070	0.230	4.000	1.380	0.910	1.050	1.000	1.370	1.010	1.120	1.000
6	1.220	6.350	1.710	1.090	1.000	0.540	0.950	2.470	1.040	1.060	1.050	1.000	0.960	1.040	0.970	1.000
7	1.200	6.380	1.680	1.090	0.990	0.490	0.940	2.470	0.980	1.040	0.990	1.000	0.950	1.040	0.960	1.000
8	1.020	0.590	0.950	1.490	1.000	0.530	0.920	1.800	1.060	1.010	1.050	1.000	1.020	1.010	1.020	1.000
9	0.920	0.860	0.910	1.820	0.900	13.860	3.440	12.650	1.090	1.020	1.070	1.000	1.060	0.980	1.050	1.000
10	1.080	1.660	1.540	3.670	1.060	0.140	0.330	5.730	1.010	1.050	1.040	0.990	1.050	1.090	1.080	0.990
11	0.900	0.050	0.090	1.100	0.850	1.350	1.320	4.890	1.190	1.030	1.040	1.000	1.210	0.030	0.090	1.000
12	0.720	0.000	0.000	1.160	0.780	0.070	0.070	15.680	0.920	0.760	0.760	1.000	0.930	0.760	0.760	1.000
13	0.930	0.730	0.870	1.000	0.810	43.850	13.710	3.620	0.910	0.950	0.920	1.000	0.890	0.940	0.910	1.000
14	0.930	0.900	0.930	0.990	0.890	8.030	1.630	3.810	0.880	0.930	0.880	0.990	0.840	0.920	0.850	0.990
15	0.820	6.530	5.190	4.480	0.970	0.620	0.700	10.190	0.950	0.640	0.710	0.990	0.930	0.630	0.700	0.990
16	0.990	0.310	0.450	7.400	1.020	6.840	5.680	1.990	0.960	1.010	1.000	0.990	0.970	1.030	1.010	0.990
17	0.880	0.230	0.390	1.070	0.810	12.320	9.410	5.800	0.940	4.640	3.710	1.000	0.920	4.790	3.810	1.000
18	1.320	0.910	0.960	1.010	0.970	0.650	0.690	10.360	0.920	0.620	0.650	0.980	0.910	0.670	0.700	0.990
19	0.780	1.130	0.980	1.020	0.780	143.440	80.500	1.020	1.020	50.190	28.500	1.000	0.990	7.790	4.790	1.000
20	0.690	0.080	0.410	4.820	0.670	0.220	0.470	10.430	1.360	1.010	1.200	1.000	1.380	1.000	1.210	1.000
21	1.100	1.550	1.400	1.000	1.030	1.480	1.320	5.190	1.070	1.330	1.240	1.000	1.020	1.090	1.070	1.000
22	1.090	0.970	0.980	0.980	1.060	0.110	0.220	2.320	1.050	0.990	1.000	0.980	1.020	1.010	1.010	0.980

продолжение на следующей странице

Продолжение таблицы 5.4

Запрос	GOO _{comb}				Top. Iter. Dec.				DPhyp				DPhyp _{mod}			
	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost	plan	exec	total	cost
23	0.630	0.530	0.600	1.190	0.640	29.360	10.020	16.840	1.000	1.050	1.010	1.000	0.970	1.030	0.990	1.000
24	0.650	2.760	1.830	1.020	0.660	1.200	0.960	5.280	1.060	1.000	1.020	0.980	1.010	1.070	1.040	1.000
25	0.980	0.670	0.870	1.110	1.000	2.630	1.580	8.660	1.060	0.730	0.950	1.000	1.020	2.160	1.420	1.000
26	1.020	1.020	1.020	1.000	1.090	24.360	14.380	22.780	1.030	0.820	0.910	1.000	1.020	1.070	1.050	1.000
27	1.080	1.020	1.030	0.980	0.940	125.820	98.210	19.970	1.010	1.030	1.030	0.980	0.990	1.050	1.040	0.980
28	1.050	4.400	3.010	1.280	1.000	0.150	0.500	3.540	1.040	1.050	1.040	1.000	0.990	1.010	1.000	1.000
29	0.830	0.840	0.830	1.130	0.710	53.410	13.470	11.750	1.040	1.090	1.050	1.000	0.980	2.430	1.340	1.000
30	1.070	1.010	1.020	0.980	0.990	131.140	103.280	19.970	1.020	1.040	1.030	0.980	0.990	1.050	1.040	0.980

Приложение Б. Результаты при истинных кардинальностях

Таблица 5.5: Результаты на наборе JOB при истинных кардинальностях: DPsize (абсолютные значения) и якорный подход (отношения к значениям DPsize). Стандартная конфигурация якорного подхода: $\tau=350$, $R_{\max}=50,000$, $\alpha=100$, $L=100,000$.

Запрос	DP standard				Якорный подход			
	plan, mc	exec, mc	total, mc	cost	plan	exec	total	cost
1a	124.7	35.0	159.7	16348	0.949	1.049	0.971	1.000
1b	120.2	33.1	153.3	16258	0.997	1.103	1.020	1.000
1c	117.6	34.7	152.3	16317	1.052	1.024	1.046	1.000
1d	117.2	33.4	150.6	16258	1.041	1.006	1.033	1.000
2a	72.5	100.0	172.5	34326	1.034	1.011	1.021	1.000
2b	72.6	96.2	168.8	32929	1.032	0.990	1.008	1.000
2c	66.4	8.7	75.0	5759	1.048	1.042	1.047	1.000
2d	72.6	155.3	227.9	52035	0.964	1.005	0.992	1.000
3a	64.8	51.7	116.5	21146	1.101	1.048	1.078	1.000
3b	68.2	33.3	101.5	19555	0.919	0.989	0.942	1.000
3c	71.1	98.4	169.5	25824	0.975	0.988	0.983	1.000
4a	122.2	33.1	155.3	20395	0.932	0.954	0.937	1.000
4b	121.3	35.6	156.9	17191	0.948	0.981	0.955	1.000
4c	121.3	39.3	160.6	23482	0.936	0.962	0.943	1.000
5a	56.7	44.0	100.7	32432	1.013	1.037	1.023	1.000
5b	55.5	42.0	97.5	34531	1.106	1.070	1.090	1.000
5c	63.9	61.5	125.5	33361	0.975	0.997	0.986	1.000
6a	96.3	7.0	103.3	3365	1.008	1.107	1.015	1.000
6b	93.5	77.9	171.4	22459	0.994	1.012	1.002	1.000
6c	87.3	3.9	91.2	2808	1.116	1.139	1.117	1.000

продолжение на следующей странице

Продолжение таблицы 5.5

Запрос	DP standard				Якорный подход			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost
6d	90.7	58.8	149.5	77225	1.106	1.015	1.070	1.000
6e	88.9	7.0	95.9	3365	1.106	1.099	1.106	1.000
6f	108.7	707.3	816.0	172147	0.976	0.937	0.943	0.806
7a	391.0	36.9	427.9	55582	1.003	1.005	1.003	1.000
7b	353.4	85.8	439.2	45062	1.034	1.078	1.042	1.000
7c	407.8	401.8	809.5	158678	0.937	0.993	0.965	1.000
8a	204.8	327.9	532.7	88929	1.111	1.010	1.049	1.000
8b	209.6	49.3	258.9	38648	1.034	1.057	1.038	1.000
8c	206.5	1832.9	2039.5	424879	1.101	0.982	0.994	1.000
8d	228.6	405.7	634.4	272843	1.034	1.030	1.031	1.000
9a	231.9	70.7	302.6	80103	1.029	1.052	1.034	1.000
9b	231.8	70.6	302.4	77556	1.037	1.032	1.036	1.000
9c	227.2	140.8	368.0	103010	1.045	1.017	1.034	1.000
9d	208.6	964.4	1173.0	337893	1.144	1.033	1.053	1.000
10a	91.4	137.7	229.1	33126	0.985	0.984	0.984	1.000
10b	84.4	59.1	143.6	30024	1.007	0.983	0.997	1.000
10c	93.7	457.7	551.5	383942	1.061	0.971	0.987	1.000
11a	157.9	11.4	169.2	3732	0.974	0.941	0.972	1.000
11b	160.2	10.5	170.7	3241	0.896	0.862	0.894	1.000
11c	143.3	29.8	173.0	7858	1.000	0.957	0.992	1.000
11d	142.5	29.9	172.4	7858	0.963	0.957	0.962	1.000
12a	268.0	81.8	349.8	39304	0.997	0.969	0.991	1.000
12b	269.5	35.8	305.3	16267	1.003	0.948	0.997	1.000
12c	282.7	262.2	545.0	81944	0.999	1.017	1.008	1.000
13a	282.3	146.8	429.1	58791	0.990	1.018	0.999	1.000
13b	302.3	58.3	360.6	48789	0.930	1.002	0.941	1.000
13c	283.1	39.7	322.8	48258	0.986	0.986	0.986	1.000
13d	290.5	564.7	855.2	130573	0.991	1.009	1.003	1.000

продолжение на следующей странице

Продолжение таблицы 5.5

Запрос	DP standard				Якорный подход			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost
14a	307.5	105.3	412.8	23792	0.979	0.966	0.976	1.000
14b	299.7	39.2	338.9	12627	1.051	1.028	1.048	1.000
14c	293.6	193.1	486.7	31317	1.016	1.026	1.020	1.000
15a	517.4	116.1	633.5	52558	0.943	1.023	0.958	1.000
15b	505.5	11.5	517.0	6497	0.946	0.998	0.947	1.000
15c	516.5	660.7	1177.2	122655	0.968	1.047	1.012	1.000
15d	481.6	681.1	1162.7	122309	1.005	0.992	0.997	1.000
16a	365.6	63.3	428.8	23743	0.993	0.970	0.990	1.000
16b	407.4	1393.8	1801.2	198788	0.990	1.015	1.009	1.000
16c	371.0	451.0	822.0	121346	1.013	0.745	0.866	0.878
16d	366.7	375.2	741.9	104473	1.091	0.751	0.919	0.942
17a	259.6	739.6	999.1	156212	1.008	0.755	0.821	0.989
17b	243.7	921.1	1164.8	149572	1.066	0.490	0.611	0.951
17c	252.6	77.0	329.6	116375	1.023	0.984	1.014	1.000
17d	242.5	851.6	1094.1	138435	1.077	0.509	0.635	0.984
17e	254.1	1095.2	1349.3	167634	1.111	1.011	1.030	1.000
17f	289.4	663.6	952.9	194599	0.942	0.999	0.982	0.840
18a	332.9	97.0	430.0	156797	1.032	1.034	1.032	1.000
18b	323.7	76.8	400.5	42128	0.977	1.018	0.985	1.000
18c	345.2	1541.7	1886.9	220043	0.966	1.015	1.006	1.000
19a	394.5	73.0	467.5	80116	0.958	1.016	0.967	1.000
19b	391.2	34.2	425.4	48129	1.016	1.035	1.018	1.000
19c	360.3	148.7	509.0	102105	1.099	1.019	1.076	1.000
19d	396.5	1440.1	1836.6	336459	0.974	1.007	1.000	1.000
20a	349.2	78.7	427.9	66704	1.016	1.021	1.017	1.000
20b	325.8	78.6	404.4	66633	1.084	1.001	1.068	1.000
20c	359.7	406.3	766.0	83439	1.028	1.012	1.020	1.000
21a	323.0	17.2	340.3	3936	1.019	1.029	1.019	1.000

продолжение на следующей странице

Продолжение таблицы 5.5

Запрос	DP standard				Якорный подход			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost
21b	320.2	10.6	330.8	3456	1.013	1.039	1.013	1.000
21c	322.1	19.2	341.3	4012	1.010	0.987	1.009	1.000
22a	563.1	139.6	702.7	29172	0.990	1.286	1.049	1.426
22b	523.2	89.8	613.0	23723	1.066	2.012	1.205	1.763
22c	526.5	188.6	715.1	38517	1.082	2.005	1.325	1.381
22d	527.7	193.6	721.3	39136	1.013	1.935	1.261	1.376
23a	557.8	70.4	628.2	46690	1.028	3.794	1.338	1.743
23b	554.6	13.0	567.6	4332	0.989	2.308	1.019	1.734
23c	555.1	132.6	687.8	51260	1.042	2.992	1.418	1.413
24a	587.1	60.0	647.1	9373	0.985	1.402	1.024	1.405
24b	587.5	10.9	598.4	6173	0.986	1.679	0.998	1.615
25a	608.0	411.8	1019.8	48874	0.973	1.017	0.991	1.000
25b	605.0	134.3	739.3	34005	0.983	1.069	0.999	1.000
25c	562.4	913.0	1475.4	81536	1.057	1.046	1.051	1.000
26a	698.1	119.6	817.7	39886	0.938	1.089	0.960	1.025
26b	693.7	32.4	726.1	8857	0.943	1.300	0.959	1.111
26c	644.9	431.8	1076.7	73017	1.134	1.042	1.097	1.163
27a	747.9	5.2	753.1	2452	0.916	1.169	0.917	1.562
27b	736.2	2.8	739.0	1400	0.975	4.493	0.989	3.331
27c	680.6	5.6	686.2	2921	1.060	4.993	1.093	1.789
28a	1040.6	82.3	1122.9	12770	0.924	1.020	0.932	1.014
28b	1036.2	59.9	1096.0	9430	0.918	1.888	0.971	1.148
28c	1137.6	89.2	1226.9	17178	0.882	1.064	0.895	1.028
29a	2408.5	7.0	2415.5	3070	0.540	5.354	0.553	17.436
29b	2560.3	6.0	2566.3	3016	0.473	5.813	0.486	17.741
29c	2424.1	18.9	2443.0	3658	0.491	13.602	0.592	41.994
30a	1076.2	266.2	1342.4	20140	0.957	1.249	1.015	2.776
30b	1076.3	95.3	1171.5	16156	0.911	0.436	0.872	2.825

продолжение на следующей странице

Продолжение таблицы 5.5

Запрос	DP standard				Якорный подход			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost
30с	1070.0	983.3	2053.3	40574	0.957	1.047	1.000	1.173
31a	930.2	88.0	1018.2	12183	0.931	1.652	0.993	1.242
31b	860.7	12.8	873.4	7061	1.063	4.651	1.115	1.660
31с	933.6	103.1	1036.7	13601	0.971	1.449	1.019	1.131
32a	34.0	4.0	38.0	2790	1.033	0.997	1.029	1.000
32b	37.9	24.7	62.6	8883	0.975	1.000	0.985	1.000
33a	555.9	12.7	568.6	3001	0.686	0.827	0.689	1.024
33b	548.7	11.4	560.1	2547	0.695	0.801	0.698	1.076
33с	460.6	13.3	473.9	3146	0.835	3.712	0.915	0.976

Таблица 5.6: Результаты на наборе JOB-Complex при истинных кардинальностях: DPsize (абсолютные значения) и якорный подход (отношения к значениям DPsize). Стандартная конфигурация якорного подхода: $\tau=350$, $R_{\max}=50,000$, $\alpha=100$, $L=100,000$.

Запрос	DP standard				Якорный подход			
	plan, mc	exec, mc	total, mc	cost	plan	exec	total	cost
1	275.8	17.2	293.0	3931	1.101	1.037	1.097	1.000
2	158.2	11.2	169.3	3734	0.980	0.970	0.979	1.000
3	320.7	18.2	338.9	3936	1.029	1.002	1.028	1.000
4	296.9	18.9	315.8	3936	1.018	0.983	1.016	1.000
5	2203.7	4955.0	7158.6	16507	0.707	0.096	0.284	1.306
6	292.0	10.4	302.4	3456	1.116	1.070	1.114	1.000
7	319.7	10.7	330.4	3456	1.023	1.020	1.023	1.000
8	1347.3	209.5	1556.9	67346	1.081	0.670	1.026	1.438
9	1047.4	80.2	1127.6	12776	0.984	1.073	0.991	1.014
10	501.7	111.1	612.8	161605	0.982	1.024	0.989	1.000
11	762.7	330.0	1092.7	48682	0.865	0.413	0.728	1.272
12	1829.1	773.9	2603.0	33789	0.778	0.635	0.735	1.030
13	979.1	193.4	1172.5	68817	0.989	1.018	0.994	1.020
14	626.9	70.0	697.0	29221	0.999	1.045	1.003	1.000
15	1119.2	679.8	1799.0	122344	1.056	1.027	1.045	1.000
16	296.4	94.5	390.9	156797	1.064	1.025	1.055	1.000
17	625.7	541.9	1167.6	157302	0.883	1.069	0.969	1.269
18	482.9	695.6	1178.4	122600	1.048	0.999	1.019	1.000
19	2571.3	154.5	2725.8	72627	0.794	1.445	0.831	1.113
20	2004.2	56.1	2060.3	7442	0.719	1.824	0.749	6.922
21	1491.6	146.4	1637.9	63986	1.091	1.457	1.123	1.133
22	455.1	269.5	724.6	131221	0.935	0.991	0.956	1.000
23	2109.7	510.2	2619.9	96298	0.721	0.943	0.764	1.104
24	4122.1	243.6	4365.7	236694	0.939	2.121	1.005	0.898
25	1633.5	477.7	2111.2	140365	0.968	0.996	0.975	0.982

продолжение на следующей странице

Продолжение таблицы 5.6

Запрос	DP standard				Якорный подход			
	plan, мс	exec, мс	total, мс	cost	plan	exec	total	cost
26	270.4	172.0	442.4	37787	1.000	1.000	1.000	1.000
27	1168.9	1676.4	2845.3	155816	0.914	0.348	0.581	0.992
28	526.0	72.2	598.1	46851	1.001	7.385	1.771	1.258
29	2044.6	423.1	2467.7	86795	0.775	1.025	0.818	1.159
30	1229.7	1678.7	2908.4	155816	0.876	0.340	0.567	0.992

Список литературы

1. Ibaraki T., Kameda T. On the optimal nesting order for computing N-relational joins // ACM Trans. Database Syst. 1984. Т. 9, № 3. С. 482–502.
2. Leis V. и др. Query optimization through the looking glass, and what we found running the Join Order Benchmark // The VLDB Journal. 2018. Т. 27, № 5. С. 643–668.
3. Karp R.M. Reducibility among combinatorial problems // Complexity of computer computations. Plenum Press, 1972. С. 85–103.
4. Moerkotte G., Fender P., Eich M. On the correct and complete enumeration of the core search space // Proceedings of the 2013 ACM SIGMOD international conference on management of data. 2013.
5. Fegaras L. A new heuristic for optimizing large queries // Proceedings of the 9th International Conference on Database and Expert Systems Applications (DEXA). 1998.
6. Moerkotte G., Neumann T. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products. 2006.
7. Moerkotte G., Neumann T. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products // Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB). 2006.
8. Moerkotte G., Neumann T. Dynamic programming strikes back // Proceedings of the 2008 ACM SIGMOD international conference on management of data. 2008.
9. Bennett K., Ferris M.C., Ioannidis Y.E. A genetic algorithm for database query

- optimization // Proceedings of the 4th International Conference on Genetic Algorithms. 1991.
10. Neumann T., Radke B. Adaptive optimization of very large join queries // Proceedings of the 2018 international conference on management of data (SIGMOD). 2018.
 11. Wehrstein J. и др. JOB-Complex: A challenging benchmark for traditional & learned query optimization // VLDB 2025 workshop: Applied AI for database systems and applications (AIDB). 2025.